

DO MORE AGENTS HELP? CONTROLLED AND PROTOCOL-ALIGNED EVALUATION OF LLM AGENT WORKFLOWS

Yuhang Fu^{1,*§} Ruishan Fang^{3,2,*} Jiaqi Shao^{4,5}
 Huiyu Zheng⁶ Zhengtao Zhu⁶ Bing Luo^{4,†} Tao Lin^{2,†}
 fuzilfuzil@bupt.edu.cn; fangruishan@westlake.edu.cn; js1139@duke.edu;
 221124120277@zjut.edu.cn; zhentaozhu@zjut.edu.cn;
 bing.luo@dukekunshan.edu.cn; lintao@westlake.edu.cn
¹Beijing University of Posts and Telecommunications ²Westlake University
³Zhejiang University ⁴Duke Kunshan University
⁵Hong Kong University of Science and Technology ⁶Zhejiang University of Technology
 *Equal contribution [†]Corresponding authors

ABSTRACT

Does adding more agents help an LLM workflow once compared systems share the same benchmark loader, tool access, answer contract, usage accounting, and trajectory logging? We introduce BenchAgent, an evaluation framework that places single-agent, fixed multi-agent (MAS), and evolving MAS workflows under one normalized execution and logging protocol. BenchAgent evaluates these substrate-internal workflows across ten reasoning, coding, and tool-use benchmarks with GPT-4.1, and separately reports a Protocol-Aligned External (PAE) GAIA study of a runtime-generated workflow. Under SI conditions, at most one of six tested MAS exceeds the matched single-agent anchor on benchmark-balanced average accuracy—EvoAgent lies within the Wilson one-run guidance—while the remaining five trail by 2.56–11.29 points and occupy more expensive accuracy–cost trade-offs. On the PAE GAIA snapshot, a Claude-Code-style runtime workflow reaches 66.72% overall and 69.23% on Level 3, more than 20 points above the strongest non-Claude baseline (Jarvis, a fixed MAS).

1 INTRODUCTION

LLM-based agent research has expanded from single reasoning-and-acting loops (Yao et al., 2022; Schick et al., 2023; Nakano et al., 2021; Yang et al., 2024) and role-specialized multi-agent systems (Li et al., 2023; Wu et al., 2023; Hong et al., 2023) to designs that configure, evolve, or generate the workflow itself (Chen et al., 2023; Yuan et al., 2024; Hu et al., 2024; Zhang et al., 2024; Fourney et al., 2024).

Accuracy alone cannot separate workflow organization from protocol advantage. Recent benchmarks broaden coverage to interactive tasks, tool use, web environments, software engineering, and diagnostic signals (Liu et al., 2023; Mialon et al., 2023; Ma et al., 2024; Zhou et al., 2023; Jimenez et al., 2024; Qin et al., 2023). Yet cross-paradigm comparisons routinely leave inputs, answer contracts, tool surfaces, usage accounting, or trajectory logging uneven across systems. For example, debate and orchestration frameworks explicitly add extra rounds, role-specialized handoffs, or tool-routing policies relative to a single-controller loop (Du et al., 2023; Wu et al., 2023; Shen et al., 2023); if such systems are compared only by final accuracy, the observed gap can mix coordination with protocol advantage. Our same-instance contrast in Figure 3 later shows a complementary failure mode: an intermediate handoff can drop a task constraint that is invisible in the final answer alone. This is why analytical traces matter in agent evaluation frameworks (Ma et al., 2024; Harbor, 2026): without them, equivalent-looking systems can spend different tokens, lose different constraints, or handle tool failures differently.

*Work was done during Yuhang’s internship in Westlake University.

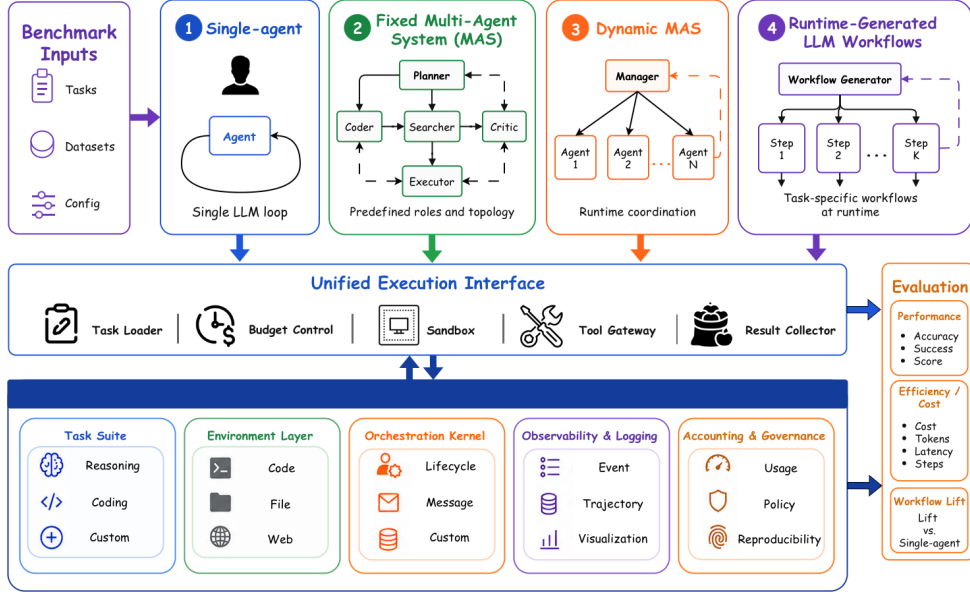


Figure 1: **BenchAgent compares workflow paradigms under a shared evaluation substrate.** Benchmark instances enter the same loading, tool-access, accounting, logging, and evaluation interfaces, while the workflow layer varies across single-agent, fixed MAS, dynamic/evolving MAS, and externally evaluated runtime-generated workflows. Within the substrate-internal setting, this design isolates workflow lift from differences in benchmark loading, tool implementation, or evaluation criteria; under the PAE setting, it documents aligned runtime workflows under the same evaluation target. The central question is whether workflow organization itself changes accuracy, cost, and trace behavior.

Figure 1 sketches the evaluation setup: benchmark instances enter a shared runtime and evaluator, while only the workflow layer varies across single-agent, fixed MAS, dynamic/evolving MAS, and externally evaluated runtime-generated execution. The goal is not to identify the strongest possible agent controller, but to estimate workflow lift: *whether reorganizing a matched controller into a MAS workflow yields reliable marginal gain under explicit protocol boundaries*. Accordingly, the paper reports one controlled substrate-internal comparison and one protocol-aligned external runtime case study rather than treating all systems as a single symmetric leaderboard.

We build BenchAgent for this purpose. BenchAgent is an evaluation framework; it wraps existing benchmarks in a common execution interface and records the process signals needed for workflow comparison. BenchAgent places single-agent systems, fixed MAS, and evolving MAS under the same benchmark-normalized interface, sandbox runtime, tool-allocation rules, and process-level logger. We use four workflow labels throughout the paper: single-agent, fixed MAS, dynamic/evolving MAS, and runtime-generated workflow. We call the first three a *Substrate-Internal* (SI) comparison when the workflow runs inside BenchAgent and exposes its messages, tool calls, usage, and termination state to the same logger. For systems that cannot be reimplemented without changing their runtime semantics, we use a *Protocol-Aligned External* (PAE) comparison: inputs, evaluator, final-answer format, tool-capability classes, backend model family, and permission profile are aligned, while controller details remain visible through retained traces.

With BenchAgent, we compare a single-agent configuration and representative fixed and evolving MAS with a GPT-4.1 backend. The broad suite covers reasoning, coding, and tool-use benchmarks (see Table 1). On GAIA (Mialon et al., 2023), we evaluate the Claude-Code-style runtime workflow (CC-workflow) through a PAE study and compare it with the BenchAgent-based MAS workflows.

The broad benchmark results show that increasing the number of agents or adding explicit coordination does not by itself guarantee positive workflow lift. Fixed and evolving MAS vary in effectiveness, token usage, and latency, and several trail the matched single-agent anchor. The GAIA comparison asks a narrower system-level question: whether a mature runtime workflow reaches a different accuracy–cost trade-off on long-horizon tool-use tasks. It does on Levels 2–3, but this is a deployed-configuration result, not evidence for any single mechanism. **Our key contributions are:**

- We introduce BenchAgent, a workflow-evaluation substrate that normalizes benchmark loading, tool access, answer contracts, usage accounting, and trajectory logging for cross-paradigm agent comparison.
- Under SI conditions with a GPT-4.1 backend, shared tools, evaluator, and logger, at most one of six tested MAS exceeds the matched single-agent anchor on average, and even that case (EvoAgent, +1.44 points) lies within the Wilson one-run guidance; the remaining five trail by 2.56–11.29 points.
- We report a PAE GAIA snapshot in which the CC-workflow reaches 66.72% overall accuracy, leads on Levels 2–3, and uses fewer recorded tokens than the strongest non-Claude baseline; isolating which design choice drives the gain requires controlled ablations beyond our deployed-configuration comparison.

2 RELATED WORK

2.1 AGENT WORKFLOW DESIGN

ReAct, Toolformer, WebGPT, SWE-agent, and HuggingGPT/JARVIS show that a single instrumented controller can already serve as a competitive workflow (Yao et al., 2022; Schick et al., 2023; Nakano et al., 2021; Yang et al., 2024; Shen et al., 2023). We therefore treat the single-agent baseline as a matched workflow anchor rather than as a bare model call or as a claim about the strongest possible single-controller design; recent work corroborates that a strong single-agent baseline can match or exceed homogeneous MAS (Xu et al., 2026).

Multi-agent systems make the division of labor explicit. CAMEL and MetaGPT define agent roles and communication procedures (Li et al., 2023; Hong et al., 2023), AutoGen provides a conversable-agent substrate for fixed or adaptive interaction patterns (Wu et al., 2023), and debate-style systems aggregate multiple proposals before finalization (Du et al., 2023; Chan et al., 2023). Li et al. (Li et al., 2024) show that sampling-and-voting improves several LLM benchmarks; DSPy (Khatab et al., 2023) instead optimizes LM pipelines through compilation. Dynamic and evolving systems search over or mutate the workflow structure within a designer-specified space, as in AgentVerse, EvoAgent, ADAS, AFlow, and Magentic-One (Chen et al., 2023; Yuan et al., 2024; Hu et al., 2024; Zhang et al., 2024; Fournery et al., 2024); MaAS extends this by searching over query-dependent agent topologies via an agentic supernet (Zhang et al., 2025); MASPO further co-optimizes agent prompts jointly across the pipeline (Wang et al., 2026). Cemri et al. (Cemri et al., 2025) catalog MAS failure modes attributable to coordination design; Kim et al. (Kim et al., 2025) further argue that net workflow gain requires coordination benefit to exceed communication overhead. These systems motivate our taxonomy, yet prior comparisons rarely run single-agent, fixed MAS, evolving MAS, and runtime workflows under a unified logging and accounting protocol.

2.2 AGENT EVALUATION FRAMEWORKS

AgentBench, GAIA, AgentBoard, WebArena, SWE-bench, ToolBench, and Silo-Bench (Liu et al., 2023; Mialon et al., 2023; Ma et al., 2024; Zhou et al., 2023; Jimenez et al., 2024; Qin et al., 2023; Zhang et al., 2026) extend agent evaluation to interaction, tool use, web/software tasks, analytical traces, and distributed coordination. Harbor (Harbor, 2026) is closest in infrastructure, evaluating and optimizing sandboxed agents and models in containerized environments. BenchAgent occupies a different scope: it introduces neither a new task dataset nor a benchmark split, but instead aligns execution interfaces, tool surfaces, usage accounting, and trajectories, so that coordination policies can be compared on existing benchmarks. Table 6 in Appendix summarizes the feature-level distinction from AgentBench and Harbor.

2.3 RUNTIME-GENERATED WORKFLOWS

Recent engineering systems push part of workflow design into the execution loop. Lang-Graph (LangChain, 2026) and CrewAI (CrewAI, 2026) provide graph orchestration and role-based crews; Anthropic (Anthropic, 2024; 2025e;d) documents subagent creation, context-window isolation, and permission-scoped delegation patterns instantiated in Claude Code (Anthropic, 2025b;a;c); similar agentic coding interfaces appear in OpenAI Codex and OpenCode (OpenAI, 2026; OpenCode, 2026). These systems are documented primarily through official documentation rather than peer-reviewed publications; we cite them as engineering descriptions, not empirical evidence. Our GAIA experiment asks whether the CC-workflow paired with a GPT-4.1 backend reaches a different

accuracy–cost profile from BenchAgent-based fixed and evolving MAS under a documented PAE comparison.

3 EVALUATION PROTOCOL

3.1 WORKFLOW LIFT AS THE MEASUREMENT TARGET

We evaluate agent systems as *workflow organizations*, not isolated products. For a benchmark instance x , a workflow w produces a final answer $\hat{y} = w(x)$ and, when available, an execution trace $\tau_w(x)$ containing model calls, tool calls, messages, artifacts, and termination events. A task evaluator $E(x, \hat{y}) \in \{0, 1\}$ measures success, while a cost summary $c(\tau_w)$ records usage signals such as tokens, latency, tool calls, and delegation structure. The comparison therefore asks not only whether $\hat{y}$ is correct, but also what trace τ_w produced it.

We distinguish workflow categories by how $\mathcal{A}_t$ (active agents), G_t (communication or delegation topology), and $\mathcal{T}_t$ (tool scopes) are specified over time:

- **Single-agent workflow.** Keeps $|\mathcal{A}_t| = 1$ throughout; a single controller handles the entire trace.
- **Fixed MAS.** Operates with a predefined $(\mathcal{A}, G, \mathcal{T})$, such as solver–critic–aggregator or debate-style handoffs.
- **Evolving MAS.** Selects or mutates G at runtime from a designer-specified family $\mathcal{G}$ (a.k.a. dynamic MAS).
- **Runtime-generated workflow.** Changes $\mathcal{A}_t$, G_t , and $\mathcal{T}_t$ during execution by creating task-specific agents, assigning private context, choosing tool scopes, or adding verification and recovery branches.

Table 10 gives the full terminology map.

Definition 1 (Workflow lift). *Our primary substrate-internal quantity is workflow lift: the change in accuracy and cost when a single-agent workflow is replaced by a fixed or evolving MAS workflow, with the base model, benchmark loader, tool interface, answer contract, evaluator, and accounting substrate held constant.*

This framing does not require the single-agent anchor to be the strongest possible ReAct implementation; instead, it requires the compared workflows to share the same control substrate, so that the measured difference reflects the tested workflow organization. Section 3.3 describes how single-run lift estimates are interpreted under Wilson guidance.

3.2 COMPARISON SETUP: SI AND PAE

BenchAgent provides the shared execution substrate for the SI comparison and normalizes six interfaces—benchmark loading, input and answer formatting, runtime control, tool access, usage and trajectory logging, and evaluator calls. In the broad-benchmark experiment (Section 4.2), Jarvis/HuggingGPT-style execution, LLM-Debate, EvoAgent, AutoGen, CAMEL, and ChatEval are implemented as workflow instances over this substrate rather than as unrelated tool stacks. This substrate renders final scores, token usage, latency, tool calls, message histories, agent identifiers, and stage-level traces comparable across workflows. Appendix A.2 includes retained prompt excerpts that document the controller instructions and output contract used for the external runtime workflow.

Claude Code cannot be reimplemented inside BenchAgent without altering its runtime semantics. The GAIA experiment (Section 4.3) therefore adopts a PAE comparison. The CC-workflow shares the GPT-4.1 backend family, GAIA validation inputs, answer-only output schema, evaluator, and GAIA-relevant tool-capability classes with the SI baselines, while internal controller details remain only partially visible through retained traces. Table 11 in the Appendix lists which protocol fields are aligned and which cannot be certified from the released artifacts.

3.3 REPORTING PROTOCOL AND WILSON GUIDANCE

For each instance, we record task success, token usage, wall-clock latency, and the full trajectory. When traces expose sufficient structure, we also derive process summaries—spawned agents, tool calls, and delegation depth—used for interpretation, not as additional success metrics. Broad-benchmark average accuracy is the unweighted mean over ten benchmark-level accuracies; GAIA overall is weighted by level sizes $(N_1, N_2, N_3) = (53, 86, 26)$. All numbers are pass@1 single-run results.

We use the *Wilson 95% binomial confidence interval* (Wilson, 1927) as a conservative scale for one-run accuracy gaps: for an observed proportion $\hat{p}$ over N instances, the interval covers the unknown success probability more reliably than the normal approximation when $\hat{p}$ is near 0 or 1 or when N is small. We report its two-sided half-width per benchmark in Table 5 and treat any pass@1 gap smaller than this half-width as descriptive rather than as stable ordering evidence. Concretely, differences below roughly 17 points on AIME ($N=30$) or 18 points on GAIA Level 3 ($N=26$) should not be read as ordering evidence under one run.

4 EXPERIMENTS

We instantiate two experiments under the protocol of Section 3. The first experiment (Section 4.2) tests whether fixed and evolving MAS produce positive workflow lift over a matched single-agent anchor on ten broad benchmarks under SI conditions; the second (Section 4.3) places a Claude-Code-style runtime workflow on GAIA under PAE alignment. Each number reported below is a pass@1 result from one end-to-end run: we treat gaps smaller than the Wilson half-widths in Appendix Table 5 as descriptive rather than as stable ordering evidence, and we do not attribute the CC-workflow’s GAIA advantage to a single mechanism without ablation evidence.

4.1 EXPERIMENTAL SETUP

Datasets. The broad SI comparison covers ten benchmarks: MATH, AIME, GSM8K, DROP, BBH, MMLU-Pro, HumanEval, MBPP, HotpotQA, and IFEval. The runtime-workflow study uses the full GAIA validation split (Mialon et al., 2023) ($(N_1, N_2, N_3) = (53, 86, 26)$, weighted by level size for overall accuracy), which stresses multi-step retrieval, file inspection, state preservation, and delayed final-answer release.

Tool allocation. Tool access is benchmark-specific but fixed across compared workflows. In the SI suite, HotpotQA uses the expanded full BenchAgent tool registry, whereas MATH, AIME, GSM8K, DROP, BBH, MMLU-Pro, HumanEval, MBPP, and IFEval expose only `python_interpreter` beyond final-answer emission. GAIA uses the corresponding full-tool regime: BenchAgent-based systems receive the expanded full registry, and the CC-workflow is matched at the level of GAIA-relevant tool-capability classes under the PAE protocol. Appendix Table 4 summarizes the benchmark-level allocation.

Compared systems. The single-agent anchor is BenchAgent Core—a full substrate run, not a bare LLM call—sharing the same loader, evaluator, and tool registry as all MAS wrappers. It serves as the workflow-lift estimator for the broad-benchmark experiment (Section 4.2), not as a claim about the strongest possible single-agent controller. Fixed MAS baselines include Jarvis (Shen et al., 2023), LLM-Debate (Du et al., 2023), AutoGen (Wu et al., 2023), CAMEL (Li et al., 2023), and ChatEval (Chan et al., 2023); EvoAgent (Yuan et al., 2024) represents the evolving MAS category. Implementation notes, transfer sanity checks, and a vanilla ReAct anchor calibration appear in Appendix Tables 21 and 7.

CC-workflow. Evaluated only on GAIA under a PAE protocol: inputs, answer schema, evaluator, backend model family, and GAIA-relevant tool-capability classes are aligned; the controller remains partially external. Full protocol documentation appears in Appendix Tables 11 and 12.

Model and infrastructure. All reported main-text BenchAgent results use GPT-4.1 (gpt-4.1-2025-04-14) (OpenAI, 2025). Dataset sampling, tool-surface summaries, GAIA-only backend-generalization checks, and trajectory evidence appear in the appendix; the backend checks are scoped to Qwen3-32B for BenchAgent-compatible GAIA workflows and GLM-5 for the BenchAgent GAIA anchor (Appendix Tables 8 and 9).

4.2 BROAD BENCHMARK RESULTS

This experiment tests whether fixed or evolving MAS produce workflow lift over the BenchAgent single-agent anchor under matched model, tool surface, evaluator, and logging protocol; results appear in Table 1, with the accuracy–cost view in Figure 2. Appendix Table 7 calibrates the anchor against a standalone vanilla ReAct controller without redefining the comparison target.

Fixed and evolving MAS do not consistently produce positive workflow lift. The matched single-agent anchor reaches 74.12% benchmark-balanced average accuracy, while EvoAgent reaches 75.56%. These broad-suite averages are descriptive benchmark-balanced means rather than interval estimates on a single shared instance pool; per-benchmark Wilson half-widths in Table 1 carry the row-level uncertainty. EvoAgent is the only MAS with numerically positive average lift, but its

Table 1: **Substrate-Internal broad-benchmark comparison of single-agent, fixed MAS, and evolving MAS under matched protocols.** Wilson 95% confidence-interval half-widths are shown for the individual benchmark rows only; the Avg. Acc. row reports benchmark-balanced average accuracy, while Avg. Tok. reports instance-level average end-to-end token usage and Avg. Time reports instance-level average execution time. Section 3.3 describes averaging and interpretation.

Benchmark / Metric	Single Agent	EvoAgent	LLM-debate	Camel	AutoGen	Jarvis	ChatEval
MATH	66.75 $\pm$ 4.6%	61.75 $\pm$ 4.7%	69.34 $\pm$ 4.5%	61.05 $\pm$ 4.8%	40.53 $\pm$ 4.8%	60.25 $\pm$ 4.8%	59.75 $\pm$ 4.8%
AIME	46.67 $\pm$ 16.8%	36.67 $\pm$ 16.3%	26.67 $\pm$ 15.1%	26.08 $\pm$ 15.0%	20.00 $\pm$ 13.9%	26.70 $\pm$ 15.1%	3.33 $\pm$ 8.0%
GSM8K	94.50 $\pm$ 2.3%	93.25 $\pm$ 2.5%	93.50 $\pm$ 2.4%	86.05 $\pm$ 3.4%	88.00 $\pm$ 3.2%	89.00 $\pm$ 3.1%	85.00 $\pm$ 3.5%
DROP	90.50 $\pm$ 2.9%	93.00 $\pm$ 2.5%	89.75 $\pm$ 3.0%	88.62 $\pm$ 3.1%	83.25 $\pm$ 3.7%	90.00 $\pm$ 3.0%	90.50 $\pm$ 2.9%
BBH	78.25 $\pm$ 4.0%	94.00 $\pm$ 2.4%	71.48 $\pm$ 4.4%	61.00 $\pm$ 4.8%	66.67 $\pm$ 4.6%	53.50 $\pm$ 4.9%	73.00 $\pm$ 4.3%
MMLU-Pro	86.50 $\pm$ 3.4%	87.00 $\pm$ 3.3%	79.50 $\pm$ 3.9%	60.11 $\pm$ 4.8%	69.50 $\pm$ 4.5%	77.50 $\pm$ 4.1%	71.00 $\pm$ 4.4%
HumanEval	84.73 $\pm$ 6.2%	81.68 $\pm$ 6.6%	93.89 $\pm$ 4.2%	89.68 $\pm$ 5.3%	61.83 $\pm$ 8.2%	88.55 $\pm$ 5.5%	86.25 $\pm$ 5.9%
MBPP	68.32 $\pm$ 4.9%	73.02 $\pm$ 4.7%	72.72 $\pm$ 4.7%	75.07 $\pm$ 4.6%	73.02 $\pm$ 4.7%	75.95 $\pm$ 4.5%	75.36 $\pm$ 4.6%
HotpotQA	62.25 $\pm$ 4.7%	68.50 $\pm$ 4.5%	65.00 $\pm$ 4.7%	55.53 $\pm$ 4.8%	58.75 $\pm$ 4.8%	64.75 $\pm$ 4.7%	60.00 $\pm$ 4.8%
IFEval	62.75 $\pm$ 4.7%	66.75 $\pm$ 4.6%	53.79 $\pm$ 4.9%	60.50 $\pm$ 4.8%	66.73 $\pm$ 4.6%	42.75 $\pm$ 4.8%	84.25 $\pm$ 3.6%
Avg. Acc.	74.12%	75.56%	71.56%	66.37%	62.83%	66.90%	68.84%
Avg. Tok.	27434.55	34153.68	36669.25	8470.19	13603.05	9668.33	105838.02
Avg. Time	106.29s	82.76s	26.63s	15.75s	14.22s	11.20s	56.08s

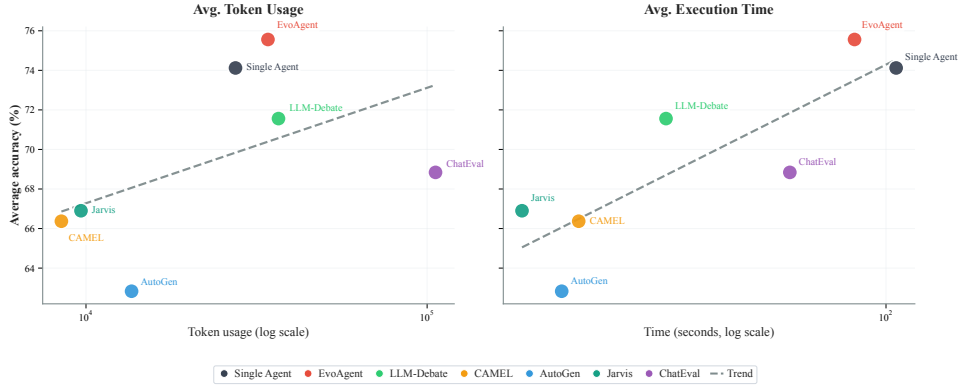


Figure 2: **Accuracy-cost and accuracy-time trade-offs under SI conditions.** Left: benchmark-balanced average accuracy against instance-level average end-to-end token usage. Right: benchmark-balanced average accuracy against instance-level average execution time. Each point represents one workflow, and the dashed line traces the empirical Pareto front under this descriptive aggregation.

Table 2: **GAIA validation pass@1 comparison under a PAE runtime-workflow setting.** Tok. reports instance-level average end-to-end token usage, and Time reports instance-level average wall-clock time.

Paradigm	Method	GAIA-L1	GAIA-L2	GAIA-L3	Avg./Overall	Tok.	Time
Single-Agent	Single Agent	58.82 $\pm$ 12.8%	30.00 $\pm$ 9.5%	19.23 $\pm$ 14.7%	37.56 $\pm$ 7.3%	459652.83	213.33s
Evolving MAS	EvoAgent	54.90 $\pm$ 12.9%	30.00 $\pm$ 9.5%	19.23 $\pm$ 14.7%	36.30 $\pm$ 7.3%	1573918.58	325.04s
Fixed MAS	LLM-Debate	52.94 $\pm$ 13.0%	34.00 $\pm$ 9.8%	15.38 $\pm$ 13.7%	37.15 $\pm$ 7.3%	414812.44	201.53s
Fixed MAS	Camel	54.90 $\pm$ 12.9%	34.00 $\pm$ 9.8%	26.92 $\pm$ 16.2%	39.60 $\pm$ 7.4%	468587.39	387.42s
Fixed MAS	AutoGen	56.60 $\pm$ 12.9%	30.00 $\pm$ 9.5%	11.54 $\pm$ 12.5%	35.64 $\pm$ 7.2%	383190.65	188.98s
Fixed MAS	Jarvis	66.03 $\pm$ 12.4%	43.02 $\pm$ 10.2%	19.23 $\pm$ 14.7%	46.66 $\pm$ 7.5%	332285.96	402.11s
Fixed MAS	ChatEval	49.02 $\pm$ 13.0%	34.88 $\pm$ 9.9%	26.92 $\pm$ 16.2%	38.17 $\pm$ 7.3%	1886517.67	443.76s
Runtime-Generated Workflow	CC-workflow	60.78$\pm$12.7%	69.62$\pm$9.5%	69.23$\pm$16.7%	66.72$\pm$7.1%	52984.69	134.90s

+1.44-point gain is smaller than the one-run uncertainty guidance; the remaining MAS fall below the anchor at 62.83%–71.56%. Holding model, tools, evaluator, and logger fixed, adding roles or handoffs did not lift overall performance. **Takeaway:** *at most one MAS exceeds the anchor on average, and that case sits within the one-run uncertainty scale.*

Cost varies more sharply than accuracy. Figure 2 shows workflows with similar benchmark-balanced accuracy but different instance-level token usage and latency. EvoAgent’s +1.44-point gain comes at higher token cost; ChatEval is far more token-intensive while trailing the baseline; Camel

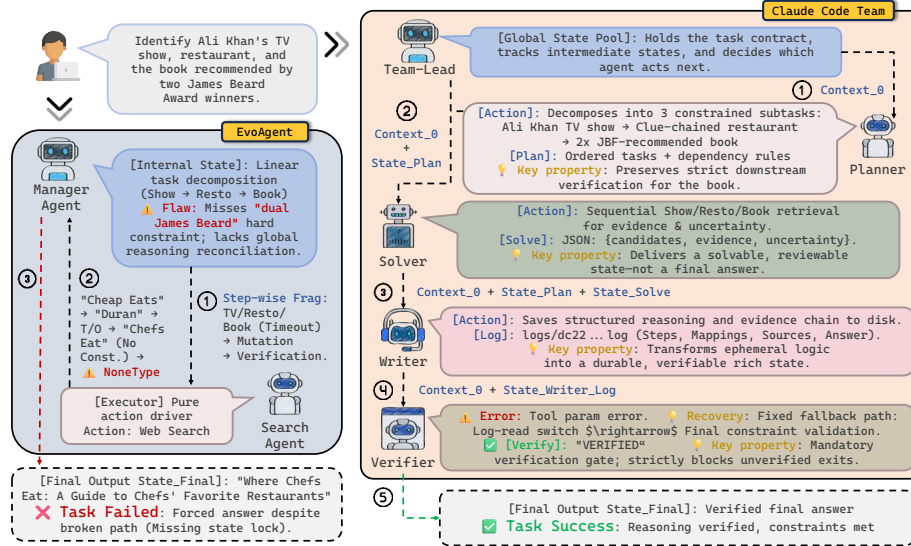


Figure 3: **Same-instance GAIA contrast: EvoAgent vs. CC-workflow.** EvoAgent loses a task-critical constraint during linear decomposition, whereas the retained CC-workflow trace preserves intermediate state and verifies before finalization.

and Jarvis are lighter and faster but lose accuracy. **Takeaway:** *accuracy-similar workflows can have sharply different token use and latency.*

Workflow structure, not agent count, explains MAS variation (Appendix Table 22). Agent count does not explain MAS behavior in our suite: EvoAgent gains on BBH through prompt-scaffold search, LLM-Debate gains on HumanEval and MATH where proposals are verifiable, and ChatEval gains on IFEval under multi-judge instruction checking. The same protocols hurt elsewhere. **Takeaway:** *aggregate MAS behavior follows task-protocol fit rather than agent count.*

MAS gains are task-dependent. EvoAgent’s BBH score (94.00% [± 2.4] vs. 78.25% [± 4.0] for single-agent) reflects task-specific scaffold selection, not a general multi-agent advantage, and raises a fairness question about prompt-search budget equity. LLM-Debate leads on HumanEval and MATH, where proposals are easy to check; ChatEval scores highest on IFEval but collapses on AIME. MAS help when their protocol matches the task error mode, and add overhead or lose information otherwise. Additional task-level views appear in Appendix Figures 7–10. **Takeaway:** *MAS protocols help when they match a task’s error mode, and hurt otherwise.*

4.3 GAIA RUNTIME WORKFLOW RESULTS

This experiment focuses on GAIA, where long-horizon tool use tests state preservation, evidence management, and recovery after partial failures; results appear in Table 2, with backend-sensitivity checks in Appendix Tables 8 and 9 and the same-instance contrast in Figure 3. The comparison remains PAE, as described in Section 3.2 and Appendix Table 11, so it compares aligned deployed configurations rather than isolating an internal mechanism.

The CC-workflow advantage scales with task length. The CC-workflow reaches 66.72% (± 7.1) overall, 20.06 points above the strongest non-Claude result (Jarvis, 46.66%). Jarvis leads on Level 1, where short retrieval chains often suffice, but the CC-workflow leads by 26.60 points on Level 2 and 42.31 points on Level 3, gaps larger than the uncertainty scale in Appendix Table 5. Retained accounting also reports fewer tokens and less wall-clock time than the strongest non-Claude baseline, though cost summaries depend on log visibility and provider-side bookkeeping. **Takeaway:** *the CC-workflow leads by 20.06 points overall, with a 42.31-point Level 3 gap.*

Retained traces expose structured runtime coordination. The CC-workflow traces show task-specific subagents, reusable evidence artifacts, and verifier-stage events before final-answer release (6.25 subagents on average, max delegation depth 2.5; Appendix Table 25). Figure 3 illustrates the contrast: EvoAgent loses a task-critical constraint and forces an answer, while the CC-workflow preserves state and verifies before finalization. **Takeaway:** *subagent count and delegation depth (6.25*

/ 2.5 max) track task structure. Appendix A.2 gives a retained event-level excerpt from the GAIA trajectory logs.

The two failure modes are structurally different. In fixed and evolving MAS baselines, a failed sub-step typically stays in the same linear handoff: a solver makes a claim, an aggregator receives a compressed version, and the final answer can be forced with a missing constraint. In retained CC-workflow traces, failures appear as local tool or state problems addressable by changing tool strategy, re-reading an artifact, or invoking a verifier. Candidate mechanisms—runtime delegation, persistent artifacts, verifier-stage control, and context management—remain confounded; Appendix Table 23 lists the needed ablations. *Takeaway: baseline failures get baked into linear handoffs; CC-workflow failures stay local and recoverable.*

5 DISCUSSION

The broad-benchmark results show task-specific failure modes, not a uniform MAS advantage. Debate-like aggregation helps when independent proposals are checkable (LLM-Debate on HumanEval and MATH), while evolutionary search helps when prompt or role variation finds a better reasoning mode (EvoAgent on BBH). The same machinery hurts when tasks require exact instruction following or tight evidence control, because handoffs can compress context and hide constraints. Table 1 is therefore a workflow-lift comparison around a shared SI anchor, not a leaderboard over all single-agent designs. Runtime-generated workflows require a different evaluation target. The CC-workflow exposes candidate mechanisms—separate context windows, permission scopes, persistent artifacts, and runtime delegation—that remain confounded with provider-side bookkeeping. Evaluations should therefore report workflow topology, context boundaries, tool scopes, and repair gates alongside final scores; otherwise, similar accuracy can hide incompatible coordination policies, costs, and failure modes.

We keep SI and PAE separate: SI isolates workflow wrappers over a common core, while PAE reports what a mature runtime workflow achieves under a documented but less internally controllable protocol. Collapsing them into one leaderboard would mix workflow lift with protocol advantage.

6 CONCLUSION

BenchAgent evaluates LLM-agent workflows under aligned execution and instrumentation. Across broad benchmarks, fixed and evolving MAS do not consistently beat the matched single-agent anchor and occupy different accuracy–cost trade-offs. On GAIA, the CC-workflow performs better on harder levels with lower retained-token and wall-clock cost under PAE, but this remains a deployed-configuration comparison rather than a mechanism estimate. We release BenchAgent and trajectory tooling for joint study of workflow generation, context management, tool scoping, and verification as properties distinct from agent count.

7 LIMITATIONS

Our comparison is controlled but not fully causal. The broad-benchmark MAS are re-instantiations, not exact reproductions; the CC-workflow is an external engineering exemplar routed to the same backend, so its advantage may combine workflow generation, context compaction, file and shell tooling, permission scoping, and provider-side bookkeeping. The results compare deployed configurations, not isolated mechanisms.

Mechanism evidence is partial: we report one-run pass@1 results and rely on retained traces that expose only part of Claude Code’s internals. Appendix Table 7 adds a vanilla ReAct calibration on MATH, BBH, HumanEval, and HotpotQA to contextualize the matched anchor. Future work should add repeated trials, controller-sensitivity checks, stricter tool-surface-matched ablations, runtime-workflow ablations, and fuller instrumentation for context packaging and recovery behavior.

REFERENCES

- Anthropic. Building effective agents. <https://www.anthropic.com/engineering/building-effective-agents>, 2024. Official engineering post. Accessed: 2026-04-08.
- Anthropic. Claude code overview. <https://docs.anthropic.com/en/docs/claude-code/overview>, 2025a. Official documentation. Accessed: 2026-03-25.
- Anthropic. Claude code. <https://www.anthropic.com/claude-code>, 2025b. Official product page. Accessed: 2026-03-25.
- Anthropic. Claude code subagents. <https://docs.anthropic.com/en/docs/claude-code/subagents>, 2025c. Official documentation. Accessed: 2026-03-25.
- Anthropic. Effective context engineering for ai agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025d. Official engineering post. Accessed: 2026-04-08.
- Anthropic. How we built our multi-agent research system. <https://www.anthropic.com/engineering/built-multi-agent-research-system>, 2025e. Official engineering post. Accessed: 2026-04-08.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do Multi-Agent LLM systems fail?, 2025.
- Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. ChatEval: Towards better llm-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*, 2023.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. *arXiv preprint arXiv:2308.10848*, 2023.
- CrewAI. CrewAI documentation. <https://docs.crewai.com/>, 2026. Official documentation. Accessed: 2026-05-16.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Chenglei Tan, Eduardo Salinas, Fabian Niedtner, Geoff Proebsting, Dina Bass, Jack Gerrits, Jacob Alber, Peter Zhang, Qingyu Zhu, Chi Zhang, Shital Shah, Ran Zhu, Erfan Al-Hossami, Huan Yang, Zahra Ashktorab, Nicholas Matsakis, Ahmed Hassan Awadallah, Chi Wang, and Saleema Amershi. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.
- Harbor. Harbor: A framework for evaluating and optimizing sandboxed agents and models. <https://www.harborframework.com/>, 2026. Official project page. Accessed: 2026-05-09.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.

- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems, 2025.
- LangChain. LangGraph overview. <https://docs.langchain.com/oss/python/langgraph/overview>, 2026. Official documentation. Accessed: 2026-05-16.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. *arXiv preprint arXiv:2303.17760*, 2023.
- Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. More agents is all you need. *Transactions on Machine Learning Research*, 2024. arXiv:2402.05120.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, and Yujiu Yang. AgentBoard: An analytical evaluation board of multi-turn llm agents. *arXiv preprint arXiv:2401.13178*, 2024.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- OpenAI. GPT-4.1 model documentation. <https://platform.openai.com/docs/models/gpt-4.1>, 2025. Official documentation. Accessed: 2026-05-09.
- OpenAI. Codex. <https://openai.com/codex/>, 2026. Official product page. Accessed: 2026-05-09.
- OpenCode. Opencode documentation. <https://opencode.ai/docs/>, 2026. Official documentation. Accessed: 2026-05-09.
- Yujia Qin, Sheng Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugging-GPT: Solving ai tasks with chatgpt and its friends in hugging face. *arXiv preprint arXiv:2303.17580*, 2023.
- Zhexuan Wang, Xuebo Liu, Li Wang, Zifei Shan, Yutong Wang, Zhenxi Song, and Min Zhang. MASPO: Joint prompt optimization for LLM-based multi-agent systems. In *Forty-Third International Conference on Machine Learning*, 2026. arXiv:2605.06623.

- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Sicong Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Jiawei Xu, Arief Koesdwiady, Sisong Bei, Yan Han, Baixiang Huang, Dakuo Wang, Yutong Chen, Zheshen Wang, Peihao Wang, Pan Li, and Ying Ding. Rethinking the value of Multi-Agent workflow: A strong single agent baseline. In *The Fourteenth International Conference on Learning Representations*, 2026. arXiv:2601.12307.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Siyu Yuan, Kaitao Chen, Jiangjie Ye, Chengwei Qin, Deqing Zhang, Wei Bi, Xiang Wang, and Xinran He. EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. In *Forty-Second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=imcyVlzpXh>.
- Jiayi Zhang, Zhaoheng Lan, Mingkai Hu, Yuan Wang, Zhiwei Liu, Fei Zhou, Jie Yan, Jiajun Xu, Yu Qiao, and Pengfei Li. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- Yuzhe Zhang, Feiran Liu, Yi Shan, Xinyi Huang, Xin Yang, Yueqi Zhu, Xuxin Cheng, Cao Liu, Ke Zeng, Terry Jingchen Zhang, and Wenyan Jiang. Silo-Bench: A scalable environment for evaluating distributed coordination in Multi-Agent LLM systems, 2026.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

Table 3: Dataset sampling protocol for the reported experiments. Splits larger than 400 instances are evaluated on a fixed-random-seed subset of 400 examples; splits with at most 400 instances are evaluated in full.

Dataset	Experiment	Evaluated instances	Sampling rule
MATH	Broad benchmark	400	Fixed-seed subset
AIME	Broad benchmark	30	Full split
GSM8K	Broad benchmark	400	Fixed-seed subset
DROP	Broad benchmark	400	Full split
BBH	Broad benchmark	400	Fixed-seed subset
MMLU-Pro	Broad benchmark	400	Full split
HumanEval	Broad benchmark	131	Full split
MBPP	Broad benchmark	341	Full split
HotpotQA	Broad benchmark	400	Fixed-seed subset
IFEval	Broad benchmark	400	Fixed-seed subset
GAIA validation	Runtime-workflow study	165	Full split; levels $(N_1, N_2, N_3) = (53, 86, 26)$

Table 4: Benchmark-specific tool regimes used in the reported experiments. Final-answer emission is shared across all runs through the evaluator/answer interface and is omitted from the regime labels.

Dataset(s)	Experiment	Tool regime
MATH, AIME, GSM8K, DROP, BBH, MMLU-Pro, HumanEval, MBPP, IFEval	Broad benchmark	<code>python_interpreter</code> only
HotpotQA	Broad benchmark	Expanded full BenchAgent tool registry
GAIA validation (BenchAgent-based systems)	Runtime-workflow study	Expanded full BenchAgent tool registry
GAIA validation (CC-workflow)	Runtime-workflow study	PAE-aligned GAIA-relevant full-capability tool surface

A EVALUATION PROTOCOL AND SUPPLEMENTARY EVIDENCE

The appendix reports protocol checks, auxiliary baseline-transfer evidence, runtime-generated workflow traces, and supplementary broad-benchmark analyses.

Appendix contents. This appendix is organized as follows:

- Appendix A.1 reports dataset sampling, tool regimes, Wilson guidance, framework comparison, ReAct calibration, backend checks, and protocol-alignment tables.
- Appendix A.2 gives retained prompt and trajectory excerpts for the runtime-workflow setting.
- Appendix A.3 reports illustrative process signals and separates observed trace evidence from mechanisms that require ablation.
- Appendix A.4 reports runtime-generated workflow trace statistics and same-task visual evidence.
- Appendix A.5 summarizes artifact use, release planning, and responsible-use notes.
- Appendix A.6 provides supplementary broad-benchmark visual analyses.

A.1 PROTOCOL AND FIDELITY CHECKS

Table 3 reports the evaluation size used for each dataset. For any source split with more than 400 instances, we evaluate a fixed-random-seed subset of 400 instances. For source splits with at most 400 instances, we evaluate the full split. The full-split evaluations below 400 instances are AIME (30), HumanEval (131), MBPP (341), and GAIA validation (165). The benchmark-balanced average accuracy in Table 1 is computed over benchmark-level scores rather than by pooling all evaluated instances.

Global model-call configuration. Unless otherwise noted, all experiments use `MAX_TOKEN_SIZE=8192`, `temperature=0.2`, and `top_p=1.0`.

Table 7 is a controller-strength calibration, not an alternative denominator for workflow lift. It changes the controller relative to BenchAgent Core, so the scores contextualize the matched anchor used in

Table 5: Approximate Wilson 95% confidence-interval half-widths for representative evaluated split sizes. Values are maximum half-widths over all Bernoulli accuracies and are used only as interpretation guidance for one-run pass@1 results, not as rerun variance estimates.

Split / Setting	Evaluated instances	Max Wilson half-width
Broad benchmarks evaluated on 400-instance fixed-seed subsets	400	± 4.9 pt
AIME	30	± 16.8 pt
HumanEval	131	± 8.4 pt
MBPP	341	± 5.3 pt
GAIA Level 1	53	± 13.0 pt
GAIA Level 2	86	± 10.3 pt
GAIA Level 3	26	± 17.9 pt

Table 6: Feature-level comparison with adjacent agent-evaluation infrastructure. The table is qualitative because the systems target different layers: AgentBench and GAIA-style benchmarks define tasks and interaction settings, Harbor emphasizes sandboxed agent/model evaluation, and BenchAgent emphasizes cross-paradigm workflow alignment over existing benchmarks.

Feature	AgentBench / task benchmarks	Harbor	BenchAgent
Existing benchmark reuse rather than new task split	partial	partial	yes
Shared answer contract across compared workflows	partial	partial	yes
Cross-paradigm runtime coverage (single-agent, fixed MAS, evolving MAS)	limited	partial	yes
Token and latency accounting under one protocol	partial	yes	yes
Structured trajectory serialization for workflow analysis	partial	partial	yes
Workflow-topology summaries (agents, edges, depth)	no / limited	no / limited	yes
Tool-surface alignment for coordination-policy comparison	partial	partial	yes

Table 7: Auxiliary controller calibration against a standalone vanilla ReAct controller under aligned BenchAgent tool regimes. MATH, BBH, and HumanEval use the `python_interpreter`-only regime, whereas HotpotQA uses the expanded full-tool regime. All rows use the same GPT-4.1 backend and evaluator pipeline as the corresponding BenchAgent anchor.

Dataset	Evaluated instances	BenchAgent anchor acc.	vanilla ReAct acc.	Δ acc.	ReAct token / successful case	ReAct time/inst.
MATH	400	66.75 $\pm$ 4.6%	78.28 $\pm$ 4.1%	+11.53 pt	9208.81	9.69s
BBH	400	78.25 $\pm$ 4.0%	85.03 $\pm$ 3.6%	+6.78 pt	4212.89	32.12s
HumanEval	131	84.73 $\pm$ 6.2%	77.95 $\pm$ 7.2%	-6.78 pt	59112.01	93.98s
HotpotQA	400	62.25 $\pm$ 4.7%	67.34 $\pm$ 4.6%	+5.09 pt	8136.38	24.19s

Table 1 rather than replacing the SI comparison, whose target is the effect of changing workflow organization while holding the BenchAgent execution substrate fixed.

Tables 8 and 9 report GAIA-only backend checks for BenchAgent-compatible workflows. The Qwen3-32B table uses the main-text GAIA metrics to compare the BenchAgent anchor with all fixed and evolving MAS rows inside the same BenchAgent protocol under the global model-call configuration. The GLM-5 table gives a focused BenchAgent backend-strength calibration on GAIA only. The external CC-workflow is intentionally excluded from these backend-swap checks because rerouting that controller to a different backend would change the PAE runtime configuration rather than produce a controlled BenchAgent-internal backend ablation.

The backend checks test whether the main workflow-level pattern is specific to one GPT-4.1 setting. They do not introduce a separate conclusion; instead, they provide scoped evidence for the same

Table 8: **GAIA validation pass@1 backend check with Qwen3-32B for BenchAgent-compatible workflows.** The backend check follows the global model-call configuration and reports the same metrics as Table 2. Tok. reports instance-level average end-to-end token usage, and Time reports instance-level average wall-clock time.

Paradigm	Method	GAIA-L1	GAIA-L2	GAIA-L3	Avg./Overall	Tok.	Time
Single-Agent	BenchAgent	47.17±13.0%	24.42±9.0%	19.23±14.7%	30.91±7.0%	269178.27	704.14s
Evolving MAS	EvoAgent	26.42±11.6%	23.26±8.8%	23.08±15.5%	24.24±6.5%	925891.35	424.01s
Fixed MAS	LLM-Debate	32.08±12.2%	12.79±7.1%	3.85±9.1%	17.58±5.8%	265544.08	80.60s
Fixed MAS	Camel	39.62±12.7%	23.26±8.8%	3.85±9.1%	25.45±6.6%	93501.17	389.01s
Fixed MAS	AutoGen	22.64±11.0%	16.28±7.8%	19.23±14.7%	18.79±5.9%	310282.87	88.51s
Fixed MAS	Jarvis	39.62±12.7%	18.60±8.2%	23.08±15.5%	26.06±6.6%	213783.77	507.48s
Fixed MAS	ChatEval	28.30±11.8%	17.44±8.0%	7.69±11.0%	19.39±6.0%	479251.38	207.24s

Table 9: **BenchAgent GAIA backend-strength calibration with GLM-5.** The table isolates the single-agent BenchAgent anchor under the GAIA protocol. Level cells report accuracy, instance-level average token usage, and instance-level average wall-clock time.

Paradigm	Method	GAIA-L1	GAIA-L2	GAIA-L3
Single-Agent	BenchAgent	81.13±10.4% / 268743.54 / 531.52s	64.71±10.0% / 418454.21 / 654.70s	40.00±17.6% / 3375974 / 810.14s

Table 10: Workflow-category terminology used in the paper. The main text uses the fixed and evolving categories consistently.

Category	Definition	Systems in this paper
Single-agent	One persistent controller maintains context, calls tools, and emits the final answer.	BenchAgent Core
Fixed MAS	Roles, topology, and communication procedure are specified before execution.	CAMEL, AutoGen instantiation, Jarvis/HuggingGPT-style wrapper, ChatEval, LLM-Debate
Evolving MAS	The system searches, mutates, or selects variants within a predefined workflow family.	EvoAgent
Runtime-generated workflow	The controller can create task-specific agents, state artifacts, tool scopes, and verification branches during execution.	CC-workflow with a GPT-4.1 backend

interpretation reported in the main experiments: MAS lift is not uniformly positive and depends on the interaction among backend reliability, task difficulty, and workflow structure. Under Qwen3-32B, the evaluated MAS rows all trail the BenchAgent anchor overall. Jarvis, Camel, and EvoAgent are closer to the anchor, but Camel’s Level 3 drop and the larger gaps for LLM-Debate, AutoGen, and ChatEval are consistent with the view that added coordination can be sensitive to harder multi-step evidence-control tasks. The GLM-5 BenchAgent calibration gives the complementary single-agent reference point, showing that changing the backend can substantially change absolute GAIA level scores while leaving the workflow-comparison question intact. Together, these checks support a bounded conclusion rather than a new claim: agent count or MAS structure alone is not a reliable source of improvement under the tested protocols.

Table 11 lists the retained protocol evidence for the GAIA comparison. Fields that cannot be certified from retained artifacts are marked explicitly rather than silently normalized.

Claude Code configuration notes. The retained setup artifacts enable experimental Agent Teams and define four named roles: Planner, Solver, Verifier, and Writer. The prompt forbids modifying project code, deleting files, or running destructive/network commands, and requires the Writer to emit final answers and logs under `outputs/` and `logs/`. No MCP or external servers are recorded for these runs. Local proxy credentials and API tokens are deliberately omitted from this paper.

Claude Code retained-log token accounting. The retained LiteLLM-style records make the visible token boundary auditable for the PAE run. We therefore report CC-workflow cost as retained-log

accounting rather than as provider-independent billing or a strict resource-budget ablation against BenchAgent. The counted usage includes model-visible instructions, the GAIA mission prompt, visible `system-reminder` context, tool schemas, tool-call JSON, subagent/delegation model calls, auxiliary title-generation calls, and tool results once returned to later prompts. Local shell execution is not tokenized until its output is fed back to the model. In the deduplicated batch, the logs contain 128 subagent model calls, 206 calls that emitted tool calls, 236 calls whose prompts include historical `tool_use/tool_result` context, and 22 title-generation calls. We did not observe an actual compaction-summary event in this batch; invisible provider-side policy remains outside the retained logs and is not used for mechanism attribution.

A.2 PROMPT AND TRAJECTORY EXCERPTS

This subsection records compact prompt and trajectory excerpts used to audit the reported runs. Table 16 summarizes the BenchAgent Core prompt contract from `bench/mas_arena/agents/prompts.yaml` and `bench/mas_arena/agents/bench_agent.py`. Tables 17 and 18 extend the record to the reproduced fixed and evolving MAS wrappers. Table 19 gives three retained single-task trajectory excerpts from the response and structured-log artifacts, and Table 20 adds further excerpts covering the ChatEval debate structure and the CC-workflow verifier-release gate. Long task text, local paths, and hidden tool schemas are shortened only for readability.

Tables 17 and 18 extend the prompt-contract record to the other compared workflows. The fixed MAS table makes the “roles and topology specified before execution” characterization from Table 10 concrete: each system names its agents, assigns distinct or homogeneous instruction sets, and fixes the communication procedure before any task input is seen. The EvoAgent table documents the base-prompt seeds and the LLM-driven crossover and mutation operations, clarifying that the evolutionary search is confined to the prompt-scaffold space while the BenchAgent execution substrate and tool surface remain fixed across all candidates.

Table 20 extends the trajectory record to the fixed and runtime-generated workflow paradigms in more detail. The ChatEval-1 trace illustrates how the three-expert debate converges toward a single answer across rounds for a GAIA book-title retrieval task, connecting directly to the token and model-call overhead reported in Table 14. The Runtime-2 and Runtime-3 traces come from sub-agent perspectives within the same CC-workflow run (`6.json`): Runtime-2 records the Solver role reasoning and forwarding answers to the Verifier, while Runtime-3 records the Writer role receiving Verifier-confirmed packets and enforcing the output contract. Together they provide the artifact-level support for the “verifier-stage control” mechanism entry in Table 23.

Table 21 reports an auxiliary transfer sanity check for selected MAS baselines. It compares official implementations or officially reported task-level numbers against matched non-BenchAgent re-instantiations on six shared datasets. These numbers are not the BenchAgent results in Table 1; the check is scoped to task-level plausibility because the original reports and our transferred runs do not always share the same underlying base model generation. Its purpose is to verify that the transferred wrappers remain in a plausible task-level range before they are placed under the controlled BenchAgent protocol.

The independent transfers fall within a small-to-moderate range of the official reference numbers, but this table is only a sanity check. The controlled evidence in the main paper comes from Table 1, where the compared systems are evaluated under the shared BenchAgent execution substrate.

A.3 ILLUSTRATIVE PROCESS SIGNALS AND MECHANISM EVIDENCE

Table 22 gives process context for broad-benchmark patterns in Table 1. Because the retained broad-benchmark artifacts do not expose a unified per-wrapper process summary, the measured-quantity column reports illustrative counts from retained GAIA traces for the same wrappers. These rows are used to interpret plausible workflow behavior, not to claim that the GAIA counts causally explain the broad-benchmark scores.

Table 23 separates observed trace signals from causal claims that require controlled ablation for the CC-workflow GAIA result.

Table 24 quantifies where errors are first introduced within each wrapper, using the binary-search failure attribution logs from retained GAIA runs. For each failed instance, the binary-search probe identifies the earliest conversation step at which the agent’s response diverges from the ground truth. Step 1 corresponds to the first agent turn (i.e., the initial generation before any iterative correction);

later steps indicate that the wrapper’s internal loop produced at least one intervening response before the error was committed or persisted.

The failure-step profile is consistent with two complementary observations. First, both wrappers commit the majority of analyzed errors at the first agent turn, suggesting that the tested wrapper structures often do not compensate for an initial-turn knowledge or retrieval gap. Second, while ChatEval’s debate loop does occasionally defer or surface an error at a later round (steps 3–13 account for 16.8% of ChatEval failures), this correction window is limited and does not translate into a net accuracy gain on GAIA relative to the simpler Jarvis handoff. This supports the bounded process reading in Table 22: in these retained runs, ChatEval’s additional coordination expands token usage without a proportionate observed error-correction benefit.

A.4 RUNTIME-GENERATED WORKFLOW EVIDENCE

Table 25 reports structured-event statistics from the retained Claude Code dumps that preserve explicit team-creation and role-message structure. “Subagents” counts distinct non-lead spawned agents, “Max Depth” is the longest directed delegation chain from the team lead, “Verifier Triggers” counts unique task-level non-idle verifier emissions, and “Explicit Repair Loops” counts verifier events that route control back to the solver or request correction.

These statistics support the narrower process claim made in the main text: the retained runtime-generated workflow traces repeatedly instantiate explicit subagents, reach delegation depth up to 4, and expose verifier-stage events in two structured runs. They do not support an exhaustive count of all recovery loops.

Table 26 extracts the team-initialization sequence from `6.json` item 17 (the first item with all four roles instantiated). The trace shows the team lead deciding at runtime how many agents to create, what roles to assign, and what prompt each role receives—a workflow structure that does not exist as a static template but is constructed during the GAIA run itself.

The team-initialization sequence illustrates two key properties of the runtime-generated workflow. First, the workflow structure (4-role pipeline with planner → solver → verifier → writer routing) is not hard-coded in any configuration file but is emitted live by the team lead in a single conversation turn, conditioned on the GAIA task batch. Second, each role’s prompt is embedded at spawn time and specializes the subagent’s behavior from the first message it receives—the verifier, for instance, is instructed from the outset to return failing answers to the solver rather than simply forward everything to the writer. This is structurally distinct from fixed MAS designs in which role assignments and routing are predetermined before any task is seen.

Table 27 continues the trace from team initialization to full execution, following one GAIA instance (`task_id: 48eb8242`, “How many nonindigenous crocodiles were found in Florida from the year 2000 through 2020?”, ground truth: 11) through each pipeline stage. The trace is drawn from `6.json` items 15, 22, 24, and 30, corresponding to the Planner, Solver, Verifier, and Writer turns respectively for this instance.

The execution trace exposes three runtime behaviors that are not present as logged state transitions in the fixed MAS instantiations evaluated here. First, inter-role communication is explicit and visible: each handoff from Planner → Solver → Verifier → Writer appears as a `SendMessage` event with a typed payload, making the state at each boundary auditable from the retained logs. Second, the verifier applies a format gate at step E7, distinguishing answer delivery from a simple pass-through chain; under the role prompt, a failed check would route control back to the Solver. Third, the Writer applies a skip-if-exists guard at step E10, preventing double-writes even if the same verified answer arrives via a separate message path. These state transitions are the trace-visible correlates of the candidate mechanisms in Table 23; they support the existence of verifier-stage control and explicit state handoff in this retained execution, while leaving their causal contribution to accuracy for ablation.

Table 28 records the full spawn-time role prompts for each of the four agents. These prompts are extracted verbatim from the `prompt` parameter of each `Agent` call in `6.json` item 17 (steps 2–5 of Table 26). Because the prompts are issued at spawn time and not updated within a run, they represent the complete behavioral contract for each role throughout the session: the only subsequent instructions each agent receives come from the shared mailbox.

Comparing Table 28 with the reproduced fixed MAS prompt contracts in Table 17 reveals a structural difference in how behavioral constraints are specified. Fixed MAS prompts (Jarvis, ChatEval, LLM-Debate, CAMEL) are written as general-purpose expert personas: they describe a type of reasoning

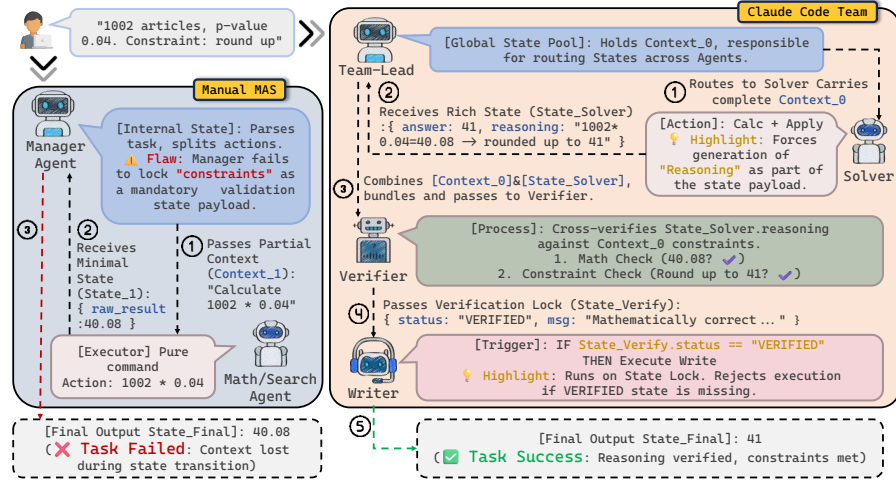


Figure 4: Controlled same-task workflow comparison between a fixed MAS pipeline and a runtime-generated workflow. The fixed MAS passes forward only a partial intermediate result and loses the instruction-level constraint, producing a locally correct computation but a globally incorrect final answer. The retained Claude trace preserves context and reasoning in the state payload and includes verifier-stage evidence before final answer release.

(mathematical, logical, critical) that applies to any task, and the role assignment does not change across instances. The CC-workflow prompts, by contrast, are operationally specific to the current batch: the Planner prompt names the exact JSONL file (`level2_part15.jsonl`) and prescribes the exact tools (`head`, `sed`, `tail`); the Solver prompt specifies the exact target for routing ("send answer, task_id, and reasoning to the Verifier"); the Verifier prompt specifies an explicit fallback path ("if not, return to Solver noting the issue"); and the Writer prompt encodes an idempotency contract ("if `outputs/<task_id>.txt` exists, SKIP"). This specificity is only possible because the team lead generates these prompts in real time, after reading the task batch and before any agent begins work.

A second distinguishing feature is the scope of the behavioral contract. In fixed MAS designs, the system-level prompt defines the full behavioral contract; agents may receive task-specific content via the conversation, but their behavioral role does not change. In the CC-workflow, the spawn-time prompt is the full behavioral contract for each role: there is no separate system-level prompt, and the only post-spawn input each agent receives is the mailbox traffic from other roles. The Solver in Table 28, for instance, is aware from its first token that it must route its output to the Verifier specifically—it does not receive this instruction as part of the task, and it cannot deviate from it without violating the prompt it was initialized with. This is the mechanism by which role specialization persists throughout a run without a central dispatcher repeatedly re-stating the routing rules.

Scope and limitations of the retained trace evidence. The trace evidence collected in Tables 26–28 and in the preceding trajectory excerpts supports a narrow set of claims. First, the team-initialization trace confirms that the CC-workflow creates its agent structure dynamically and assigns role-specific prompts at the point of instantiation, not from a fixed template. Second, the task-execution trace confirms that role-to-role handoffs are mediated by explicit `SendMessage` events, that the verifier stage receives the answer before the writer stage receives it, and that the writer enforces an idempotent write guard. Third, the spawn-time prompt table confirms that behavioral specialization for this particular GAIA batch is embedded in the initial prompt for each agent, not relayed through subsequent messages.

The retained traces are intentionally used for bounded process claims. The detailed tables are drawn from one GAIA batch partition (`level2_part15.jsonl`), so they establish that this runtime-generated structure occurred, not that the same four-role structure is universal across all CC-workflow batches. Because the team lead generates structure conditioned on the task batch, different inputs could yield different role counts or routing graphs. Trace visibility is also partial: spawn-time prompts and inter-role `SendMessage` events are visible, while provider-side bookkeeping such as context

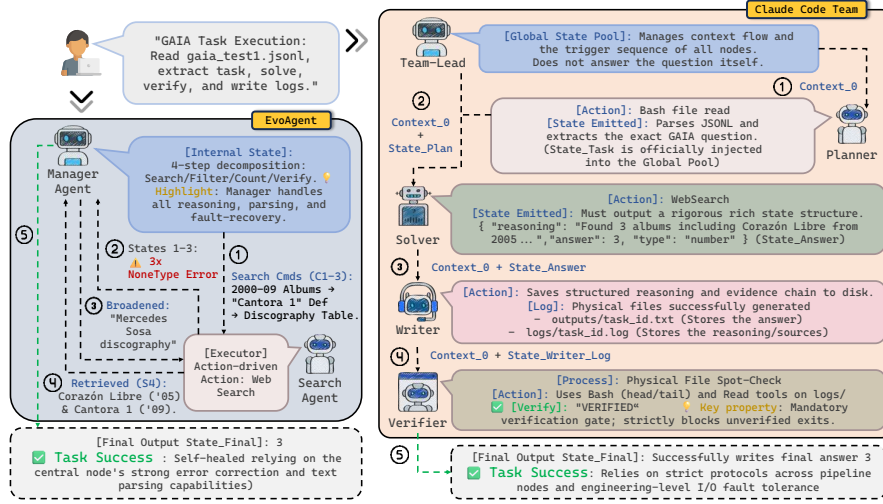


Figure 5: Supplementary matched-success GAIA case. Both EvoAgent and Claude Code eventually solve the task, but Claude Code externalizes intermediate state and routes it through verification more explicitly.

compaction, token caching, and background retry logic is outside the retained artifacts. The “explicit repair loop” count in Table 25 is therefore a lower-bound event count, not a claim that no unobserved repair behavior occurred.

The evidence tables therefore support the descriptive claim in the main text: the retained CC-workflow traces instantiate explicit subagents, implement a verifier-stage gate, and preserve routing state through visible handoffs. They do not assign the accuracy advantage to one mechanism. In particular, the logs alone cannot separate the effects of the verifier gate, the skip-if-exists idempotency guard, spawn-time prompt specificity, context packaging, and other runtime behaviors. Controlled ablations that disable individual design choices while holding all others fixed would be required for that causal attribution, as noted in Table 23.

A.5 RESPONSIBLE RESEARCH AND ARTIFACT USE

Artifact use. This paper uses public benchmarks and baseline descriptions for research evaluation, with dataset sampling, tool regimes, protocol alignment, and retained trace evidence documented in Appendices A.1–A.4. We follow the original license and access terms for the benchmarks, baselines, and tool artifacts used in the study, and we do not redistribute restricted source data, credentials, private paths, or provider tokens.

Release plan. During anonymous review, we do not publicly release non-anonymized code, local configuration, or raw traces that could expose private infrastructure, credentials, local paths, or non-public runtime details. Upon acceptance, we plan to release the evaluation code, analysis scripts, anonymized or non-sensitive aggregate statistics, and trajectory tooling needed to reproduce the reported tables and audit the retained process evidence where redistribution is permitted.

Risks and AI assistance. The main empirical risks are the one-run pass@1 setting, the PAE nature of the CC-workflow comparison, and the possibility that agent-count summaries obscure workflow, tool-surface, and context-management effects; these risks are discussed in the main-text limitations and mechanism-evidence notes. AI assistants were used for language editing, code assistance, and drafting support. The authors reviewed and take responsibility for the final claims, evidence, and manuscript text.

A.6 ADDITIONAL BROAD-BENCHMARK ANALYSES

Appendix Figures 6–10 collect the supplementary broad-benchmark views. Figure 6 summarizes the three aggregate rows used in the main-text cost discussion, while Figures 7–10 show task-level accuracy, workflow lift, per-benchmark rankings, and per-workflow benchmark-level lift profiles.

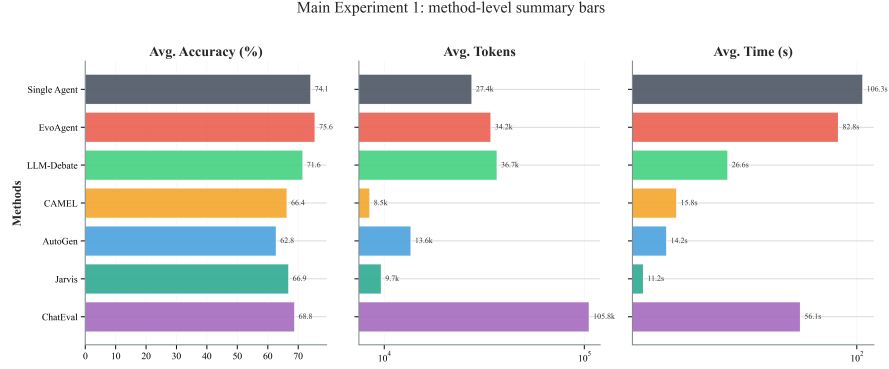


Figure 6: Main Experiment 1 method-level summaries. The panels visualize the summary rows of Table 1: benchmark-balanced average accuracy, instance-level average end-to-end token usage, and instance-level average execution time.

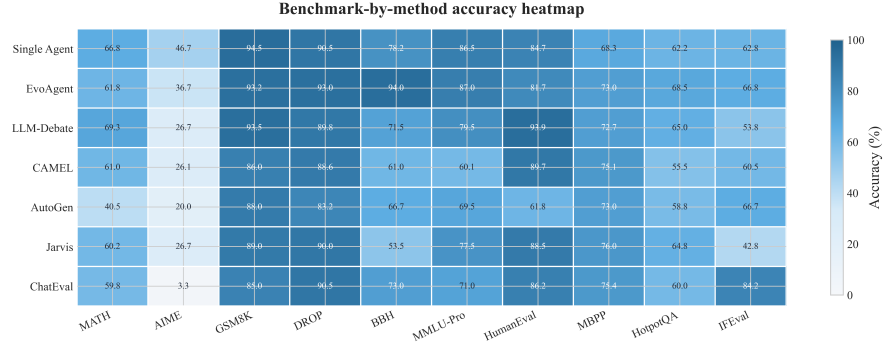


Figure 7: Benchmark-by-method accuracy heatmap for Main Experiment 1. Each cell reports benchmark-level pass@1 accuracy for one workflow on one benchmark.

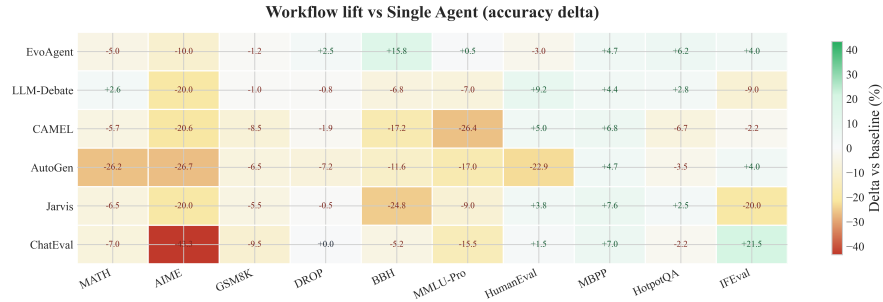


Figure 8: Workflow lift relative to the Single Agent baseline in Main Experiment 1. Green cells indicate positive accuracy delta, and orange/red cells indicate lower accuracy than the baseline on the same benchmark.

Table 11: Artifact-grounded protocol summary for the GAIA comparison.

Field	BenchAgent-Based Systems	Claude Code Configuration
Backbone and provider	Official OpenAI gpt-4.1-2025-04-14 snapshot via the provider family used in the main experiments	Official OpenAI gpt-4.1-2025-04-14 snapshot routed through the same LiteLLM-compatible proxy family; model names appearing only in setup examples are not treated as experiment settings
Model-call parameters	Global MAX_TOKEN_SIZE=8192; temperature=0.2; top_p=1.0	Global MAX_TOKEN_SIZE=8192; retained records show temperature=0.2 and top_p=1.0 for OpenAI-compatible calls
Invocation and version	BenchAgent runner scripts under the shared evaluation harness	Claude Code VS Code plugin, version 2.1.68; release date is not certified from retained artifacts
Reporting protocol	pass@1, one end-to-end run per instance	pass@1, one end-to-end run per instance
Evaluation split	GAIA validation, Levels 1–3; $(N_1, N_2, N_3) = (53, 86, 26)$	GAIA validation, Levels 1–3; $(N_1, N_2, N_3) = (53, 86, 26)$
Execution limits	manager max_steps=15; search max_steps=10; 600s API timeout; 300s default MCP/server timeout	retained artifacts show prompt-level batching and a 60s shell-command timeout; no uniform controller-level turn, agent-count, wall-clock, or provider-timeout cap is certified, so the PAE comparison is not a resource-budget-matched ablation
Tool and server configuration	expanded full BenchAgent tool registry for GAIA	Claude Code built-in tools plus Agent Teams orchestration; no MCP or external servers were enabled
Permission and sandbox surface	task-level sandbox, tool allowlists, and MCP/server timeouts controlled by BenchAgent	documented VS Code permission profile with interactive prompts bypassed for unattended execution; prompt and settings constrain intended writes to outputs/** and logs/** and deny secrets or destructive deletion
Retry and stopping	model-call retry with max_retries=3; possible suggestion-guided rerun after failed checks; termination on final answer, step budget, or runtime error	no explicit provider auto-retry field is observed; repeated/resumed calls are counted when present; final extraction uses answer-only outputs/task_id.txt files, with existing outputs skipped
Token accounting	token usage is averaged over all attempted instances from recorded prompt/completion metadata aggregated across agent messages	usage records expose prompt/completion metadata for main, subagent, tool-emitting, and auxiliary calls; visible instructions, prompts, tool schemas, tool calls, and returned tool results are counted when model-visible
Process evidence	shared BenchAgent execution substrate and structured traces	four of nine retained dumps preserve explicit team-creation, agent, and send-message events, role-scoped subagents, and verifier-stage release; per-subagent internal context is summarized through retained model-call and message records rather than fully replayable

Table 12: GAIA-critical capability and tool-surface alignment for the main comparison. BenchAgent retained counts are reconstructed from `log.rar` and the selected Jarvis/ChatEval response-log archive; fuller method-level counts appear in Tables 13 and 14.

Capability	BenchAgent Configured Surface	BenchAgent Retained Evidence	Claude Code Evidence
Web search	Web-search and retrieval tools in the GAIA registry.	<code>log.rar</code> : 11,050 <code>Request URL</code> events (17.40/inst.). Selected ChatEval: 1,836 retrieval-step calls and 3,544 executed retrieval records.	Schema exposes <code>WebSearch</code> ; retained structured events include 3 calls.
Web page fetching / browsing	Web, crawler, browser, and page-reader tools.	Selected ChatEval: 3,916 executed external-tool records (24.02/inst.); browser/crawl/archive tools appear among top executed tools.	Web-fetch style capability is visible; parser does not reconstruct every page event.
Local file discovery	Benchmark-visible files and file readers.	Discovery is not separated from benchmark file injection in retained BenchAgent logs.	2 retained <code>Glob</code> calls.
Local file reading	Text, Markdown, document, and file readers.	Selected ChatEval: 168 executed file-tool records (1.03/inst.); Jarvis and <code>log.rar</code> do not serialize typed file reads consistently.	29 retained <code>Read</code> calls; additional inspection often goes through <code>Bash</code> .
Shell and code execution	Python/interpreter-style execution through the shared runtime.	<code>log.rar</code> : 159 <code>Code Agent Step</code> and 59 <code>Executing code</code> markers. Selected ChatEval: 2,787 structured code-tool calls.	207 retained <code>Bash</code> events.
Structured tables	CSV and sheet-style extraction tools.	CSV/sheet inspection is included in selected ChatEval file-tool records; complete per-format counts are not certified.	Structured files are handled through file tools, shell commands, or Python; no specialized spreadsheet parser is certified.
Document / text parsing	Text and document parsing paths.	Text extraction and Markdown conversion appear among selected ChatEval top tools; legacy logs do not replay parser internals.	Direct file reads and shell-based parsing are visible; parser internals are not fully replayable.
Writing artifacts	Answer, trajectory, and usage serialization.	Selected response files cover all 328 Jarvis/ChatEval instances and serialize messages, usage, and final answers; <code>tool_calls</code> are absent.	65 retained <code>Write</code> events; prompt restricts writes to <code>outputs/</code> and <code>logs/</code> .
Team orchestration	Fixed, dynamic, or debate-style MAS wrappers.	Selected responses: ChatEval has 1,878 model calls (11.52/inst.); Jarvis has 165 (1.00/inst.).	7 <code>TeamCreate</code> , 31 <code>Agent</code> , and 35 <code>SendMessage</code> events.
Task/state management	Manager state, step budgets, messages, tool calls, and evaluator outputs.	Selected responses: 3,756 ChatEval messages (23.04/inst.) and 330 Jarvis messages (2.00/inst.).	Role-scoped messages, written artifacts, final-answer files, and <code>TodoWrite</code> schema are visible.
External servers	Optional MCP/server-backed registered tools.	Retained logs do not separate MCP/server-backed calls from ordinary registered tool calls.	No MCP or external servers were enabled.

Table 13: GAIA `log.rar` structure and retained event counts for BenchAgent-based systems. Each cell reports total count with mean per retained instance in parentheses. These legacy logs do not serialize structured `STEP_LOG` or `TOOL_CALL` `name=...` records, so counts are conservative retained-log evidence rather than complete hidden-controller tool traces. “Code-agent steps” counts logged `Code Agent Step` markers; “Direct external” counts `Calling tool: ...` markers after excluding `final_answer`; “Wiki/API req.” counts logged `Request URL` events; “Code exec.” counts `Executing code`; and “Tool errors” counts logged tool-execution errors. For Jarvis and ChatEval, Table 14 separately reports the selected-batch response/log parser output used for the GAIA table.

Method	Retained instances	Code-agent steps	Direct external	Wiki/API req.	Code exec.	Tool errors
Single Agent	127	45 (0.35)	116 (0.91)	2301 (18.12)	33 (0.26)	7 (0.06)
EvoAgent	127	50 (0.39)	0 (0.00)	2429 (19.13)	0 (0.00)	15 (0.12)
LLM-Debate	127	15 (0.12)	0 (0.00)	1987 (15.65)	2 (0.02)	9 (0.07)
Camel	127	21 (0.17)	10 (0.08)	2433 (19.16)	7 (0.06)	7 (0.06)
AutoGen	127	28 (0.22)	18 (0.14)	1900 (14.96)	17 (0.13)	10 (0.08)

Table 14: Selected GAIA response/log statistics for Jarvis and ChatEval. The six selected result JSON files in the supplementary archive define the evaluated batches; matching response files cover all 165 Jarvis and 163 ChatEval retained instances. Response files reliably serialize messages, model-call usage, and token usage, but they do not persist `tool_calls`. Tool-use evidence is therefore extracted from logs. “Step external” counts structured `STEP_LOG` tool calls excluding `final_answer`; “executed external” counts tool-manager `TOOL_CALL` records excluding `final_answer`; and “direct/code markers” counts unstructured `Calling tool: ... lines` excluding `final_answer` plus `Executing code: markers`. Jarvis logs do not serialize structured `STEP_LOG` or `TOOL_CALL` records for these batches, so only unstructured markers are available for Jarvis.

Method	Selected instances	Matched responses	Model calls	Step external	Executed external	Direct/code markers
Jarvis	165	165 (1.00)	165 (1.00)	0 (not serialized)	0 (not serialized)	22 (0.13)
ChatEval	163	163 (1.00)	1878 (11.52)	4623 (28.36)	3916 (24.02)	1189 (7.29)

Table 15: Retained Claude Code GAIA prompt and output contract. The table records the artifact content used for the reported runs; credentials and local proxy details are intentionally excluded.

Prompt Component	Retained Content / Operational Effect
Autonomy rule	The prompt instructs the system to operate autonomously as an Agent Team and continue without waiting for user confirmation unless a critical error occurs.
Team roles	Four named roles are specified: Planner, Solver, Verifier, and Writer. Planner inspects the project and task split; Solver reasons over each task; Verifier checks the proposed answer; Writer emits final answer files and logs.
Input-reading rule	The GAIA prompt asks the Planner to inspect JSONL split files using shell-visible file operations such as <code>head</code> , <code>sed</code> , and <code>tail</code> , rather than relying on an implicit hidden reader.
Split and batching	Retained prompts include both a first-10-task batch contract and a GAIA validation contract over <code>split_validation/gaia1_2.jsonl</code> ; existing output files are skipped rather than overwritten.
Write scope	The prompt restricts writes to <code>outputs/</code> and <code>logs/</code> . It explicitly forbids modifying project code, task data, prompts, source directories, configuration files, or existing benchmark assets.
Command restrictions	Destructive file deletion and network-download commands are forbidden by prompt-level rules; the retained settings also deny secrets and destructive deletion patterns.
Final extraction	Final answers are written as concise answer-only <code>outputs/<task_id>.txt</code> artifacts, with per-task reasoning or evidence placed under <code>logs/</code> . The user-facing final report lists processed, skipped, and generated outputs.

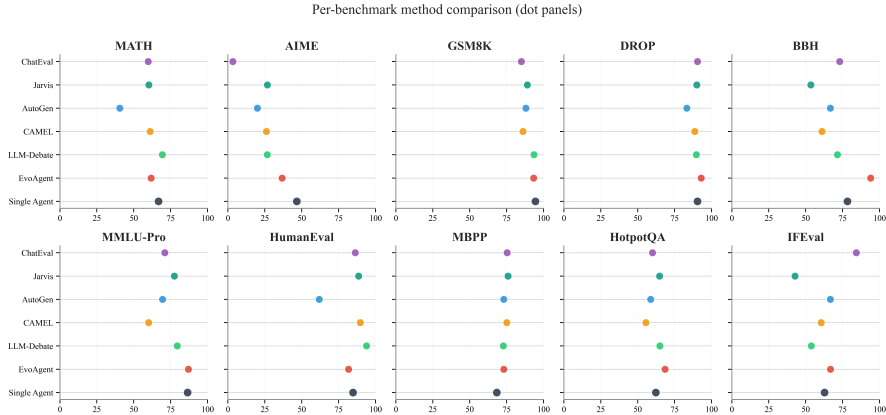


Figure 9: Per-benchmark method comparison for Main Experiment 1. Each panel shows the pass@1 accuracy distribution across workflows for one benchmark, making task-specific outliers and near-ties easier to inspect than in the wide table.

Table 16: BenchAgent Core prompt and execution contract. These fields are copied or closely paraphrased from the retained prompt templates and runner code, rather than inferred from results.

Prompt / code field	Retained wording or behavior	Evaluation role
Task wrapper	“Answer this question correctly. You have all the tools needed to find the right answer.”	Gives every benchmark instance the same task-facing wrapper before workflow-specific execution begins.
Tool priority	“Always prioritize using available tools over writing custom code.”	Prevents compared systems from silently bypassing the tool surface through ad hoc local code when a registered tool is intended.
File-access rule	“If the task involves files or attachments, you MUST use appropriate tools to access them.”	Keeps file-bearing GAIA and document tasks inside the logged tool interface rather than hidden file reads.
Search delegation	The manager can call <code>search_agent("Your detailed request")</code> for web research and online documents.	Separates manager reasoning from delegated web research while preserving a recorded hand-off.
Direct answer contract	The manager must call <code>final_answer("Your answer")</code> and keep final answers concise.	Normalizes final-answer extraction across benchmarks and workflows.
Runtime limits	The manager uses <code>max_steps=15</code> and the search agent uses <code>search_max_steps=10</code> unless configured otherwise.	Makes step budgets explicit for the SI comparison.
Verification pressure	The task wrapper says failures or <code>None found</code> are not acceptable and asks the agent to verify when needed.	Encourages evidence checking without changing the final-answer-only evaluator.
Trace serialization	Returned records include message history, final answer, manager steps, search-agent steps, and usage metadata when available.	Provides the process evidence used in the main text and appendix diagnostics.

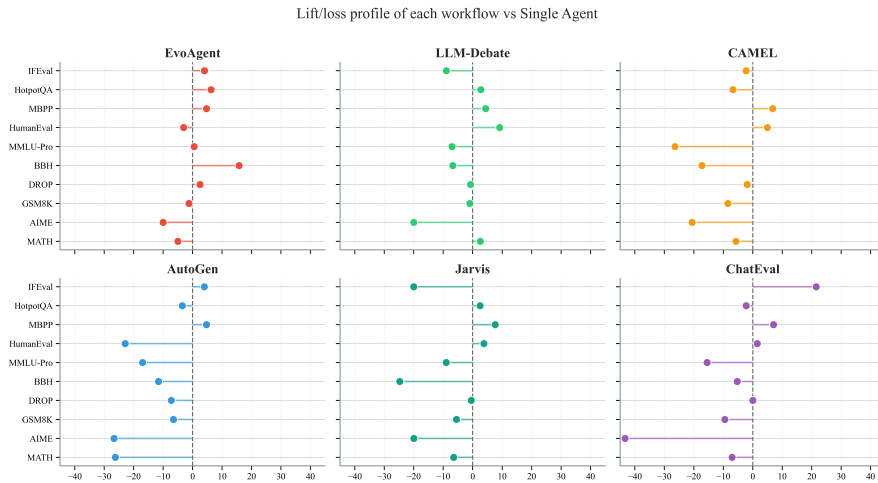


Figure 10: Per-workflow benchmark-level lift profiles relative to the Single Agent baseline. Points to the right of zero indicate positive benchmark-level accuracy delta, while points to the left indicate lower accuracy than the matched baseline.

Table 17: Retained agent role and execution contracts for the four reproduced fixed MAS wrappers. Instructions are copied or closely paraphrased from the retained source code rather than inferred from results. “LLM-only” indicates no external tools beyond the `final_answer` interface.

System	Agent instances	Role-level instruction content	Communication and termination	Tools and limits
Jarvis	Single core manager	(1) Analyze & Plan: briefly outline steps before writing code or calling tools. (2) Execute: implement the plan using Python interpreter and search tools proactively. (3) Verify & Summarize: check results and provide a concise final answer.	BenchAgent single-step execution; no inter-agent handoff; <code>final_answer</code> tool terminates.	Python + web search; <code>max_steps=15</code>
ChatEval	Math Expert (tools); Logic Expert (LLM-only); Critical Thinking Expert (LLM-only); ResultExtractor	Math Expert: analyze mathematically; verify with Python; use <code>web_search</code> for current data; use file tools for attachments. Logic Expert: analyze logical structure and implicit conditions; be extremely concise; no tools. Critical Thinking Expert: analyze from multiple angles; identify traps; be extremely concise; no tools.	Sequential per-agent contributions each round; all agents see the shared debate history; ResultExtractor aggregates the debate into a single final answer.	Math Expert: Python + web search; Logic/Critical: LLM-only
LLM-Debate	2 homogeneous BenchAgents (<code>debate_agent</code>) + Aggregator	Both agents receive the same base instruction: “You are a helpful AI assistant.” No persona differentiation. Aggregator synthesizes the debate history.	Multi-round debate (<code>rounds_num=3</code>); agents alternate; aggregator emits final answer after the last round.	BenchAgent-configured tool surface per agent
CAMEL	User role (instruction giver) + Assistant role (task executor)	User: evaluates and gives feedback; terminates with <code>TASK_FINISHED</code> when satisfied. Retained prompt: “If you feel the assistant’s response is satisfactory, please include ‘TASK_FINISHED’.” Assistant: “Respond with clarity and accuracy.”	Iterative user/assistant conversation; exits on <code>TASK_FINISHED</code> signal; synthesizer extracts answer.	BenchAgent-configured tools for Assistant

Table 18: Retained EvoAgent evolutionary search contract. The search is confined to the prompt-scaffold space; the BenchAgent ReAct core, tool surface, step budget, and `final_answer` interface are held fixed across all candidate agents throughout the evolutionary search.

Field	Retained content
Initial population	3 agents (EVO-1, EVO-2, EVO-3), each seeded from a distinct base-prompt template.
Seed prompt EVO-1	“You are a mathematics expert, skilled in solving mathematical problems. Please think step by step and solve the problem.”
Seed prompt EVO-2	“You are a logical reasoning expert, skilled in analyzing problems and finding solutions. Please provide detailed reasoning process.”
Seed prompt EVO-3	“You are a problem-solving expert, skilled in breaking down complex problems into simple steps. Please clearly show your thinking process.”
Crossover	An LLM call receives both parent system prompts and generates a new prompt that “combines the strengths” of each parent; the offspring agent inherits the merged reasoning emphasis.
Mutation	An LLM call receives one parent prompt and is asked to “create a mutated agent configuration that is different from the parent but still effective”; the output must include a <code>system_prompt</code> field.
Selection	Agents achieving better task-level performance are retained as parents for subsequent crossover and mutation.
Fixed substrate	BenchAgent ReAct core, registered tools, API settings, <code>max_steps</code> budget, and <code>final_answer</code> contract are identical across all candidate agents; only the <code>system_prompt</code> field varies across the evolutionary population.

Table 19: Three representative retained single-task trajectory excerpts. Rows preserve within-task order and report the artifact-visible event rather than a reconstructed ideal workflow.

Trace	Step	Visible actor	Retained single-task event excerpt
Core-1	1	user wrapper	The retained query asks the Girls Who Code percentage-change question under the shared BenchAgent wrapper: use tools if needed, verify when needed, and return the answer directly. Source artifact: <code>bench_agent_7d4a...</code>
Core-1	2	final-answer event	The final-answer event emits 22 with recorded usage metadata; the retained result is correct against the ground truth 22.
Core-2	1	user wrapper	A text-grid task asks the agent to read all letters left-to-right from a 5-by-7 block and recover the sentence. Source artifact: <code>jarvis_50ad...</code>
Core-2	2	final-answer event	The final-answer event emits THE SEA GULL GLIDED PEACEFULLY TO MY CHAIR, matching the expected sentence up to capitalization and spacing.
Runtime-1	1	team-lead TeamCreate	/ The controller creates a GAIA validation team for <code>split_validation/level2_part7.jsonl</code> , with planner, solver, verifier, and writer roles. Source artifact: <code>3.json</code> .
Runtime-1	2	team-lead Agent	/ The planner is spawned with a bash-only JSONL extraction instruction and asked to return task ids and raw question lines.
Runtime-1	3	team-lead / Bash	The retained command <code>head -n 3 level2_part7.jsonl</code> exposes the source-task read; the first visible task id begins <code>a7feb290...</code>
Runtime-1	4	team-lead SendMessage	/ The lead sends the planner a follow-up to return parseable JSON lines and skip tasks with existing outputs/ <code><task_id>.txt</code> , making the handoff and output policy explicit.

Table 20: Additional retained trajectory excerpts for the fixed and runtime-generated workflow paradigms. ChatEval-1 comes from the extracted GAIA debate logs; Runtime-2 and Runtime-3 come from two sub-agent perspectives in the same `6.json` CC-workflow dump. Rows preserve within-task order; long content, paths, and tool schemas are shortened for readability.

Trace	Step	Visible actor	Retained event excerpt
ChatEval-1	1	user wrapper	A GAIA book-title retrieval task is routed to the three-expert debate team under the shared BenchAgent task wrapper. Source artifact: <code>chateval_newcore_gaia_search</code> debug log.
ChatEval-1	2	Math Expert / Round 1	Math Expert performs a web search and retrieves: “[1] James Beard Award-winning food journalists Jane and Michael Stern published a book titled ‘500 Things to Eat Before It’s Too Late’ in which the restaurant is recommended.”
ChatEval-1	3	Logic Expert / Round 1	Logic Expert emits the minimal answer form “the book title only,” relying on the Math Expert’s search evidence without issuing a redundant tool call.
ChatEval-1	4	Critical Thinking Expert / Round 1	Critical Thinking Expert emits the normalized form: “should be: ‘five hundred things to eat before it’s too late.’ ”
ChatEval-1	5	All agents / Round 2	All three experts converge on “five hundred things to eat before it is too late.” The ResultExtractor aggregates the debate history into a single final-answer event.
Runtime-2	1	Solver / role init	Solver role receives its scope: for each incoming task (<code>task_id</code> , question), solve strictly from the question and attached context, format the answer per GAIA requirements, and send only <code>task_id</code> , answer, and reasoning to the Verifier. Source artifact: <code>6.json</code> (sub-agent perspective).
Runtime-2	2	Solver / teammate-message	Planner forwards task <code>48eb8242...</code> : “How many non-indigenous crocodiles were found in Florida from the year 2000 through 2020? (USGS Nonindigenous Aquatic Species database.)”
Runtime-2	3	Solver / SendMessage	Solver sends to Verifier: “Answer: 11; Task ID: <code>48eb8242...</code> ; Reasoning: According to the USGS Non-indigenous Aquatic Species database, there were 11 confirmed occurrences of nonindigenous crocodiles in Florida from 2000 through 2020.”
Runtime-2	4	Solver / teammate-message	Planner forwards task <code>c8b7e059...</code> : “The work referenced in footnote 397 of Federico Lauria’s 2014 dissertation is also the source of two Smithsonian paintings. What is the sum of the paintings’ years?”
Runtime-2	5	Solver / SendMessage	Solver sends to Verifier: “Answer: 23; Task ID: <code>c8b7e059...</code> ; Reasoning: Footnote 397 of Federico Lauria’s 2014 dissertation references Dante’s <i>Inferno</i> ; the two Smithsonian paintings are ‘The Fifth Circle’ and ‘The City of Dis’; 23 was computed from the associated publication years.”
Runtime-3	1	Writer / role init	Writer role receives team scope: check for existing <code>outputs/<task_id>.txt</code> before writing; write only verified answer packets forwarded by the Verifier; save full reasoning to <code>logs/<task_id>.log</code> . Source artifact: <code>6.json</code> (sub-agent perspective).
Runtime-3	2	Writer / teammate-message	Verifier forwards confirmed packet for task <code>48eb8242...</code> : “Answer: 11; Reasoning: According to the USGS Nonindigenous Aquatic Species database, there were 11 confirmed occurrences...” The message is gated by the Verifier’s format and correctness check before reaching the Writer.
Runtime-3	3	Writer / Read	Writer checks whether <code>outputs/48eb8242...txt</code> already exists before writing, enforcing the skip-existing output policy from the prompt contract.
Runtime-3	4	Writer / teammate-message	Verifier forwards confirmed packet for task <code>c8b7e059...</code> : “Answer: 23; Reasoning: Footnote 397 of Federico Lauria’s 2014 dissertation references Dante’s <i>Inferno</i> ...”
Runtime-3	5	Writer / Write	Writer writes the verified answer to <code>outputs/c8b7e059...txt</code> ; the Verifier’s confirmation is the necessary precondition for any write event, implementing the verifier-release gate visible in the <code>6.json</code> team <code>run6</code> statistics.

Table 21: Auxiliary transfer sanity check for selected representative baselines on six shared datasets. Scores are percentages. The independent transfer rows are separate non-BenchAgent checks and are not the Table 1 results. “Mean Abs. Diff.” and “Max Diff.” are absolute percentage-point differences over MATH, AIME, DROP, MMLU-Pro, BBH, and HumanEval; they are range diagnostics for transfer plausibility rather than code-identity estimates.

Method	Setting	MATH	AIME	DROP	MMLU-Pro	BBH	HumanEval	Avg.	Mean Abs. Diff.	Max Diff.
EvoAgent	Official	68.00	16.67	88.00	60.00	75.00	72.28	63.32	6.79	12.72
	Independent transfer	65.00	26.70	80.00	60.00	82.00	85.00	66.45		
Jarvis	Official	59.00	6.67	85.00	55.00	55.00	92.00	58.78	2.89	5.00
	Independent transfer	55.00	10.00	85.00	50.00	54.00	88.00	57.00		
LLM-debate	Official	53.00	13.30	84.00	66.00	79.00	88.00	63.88	2.90	3.37
	Independent transfer	50.00	16.67	87.00	63.00	82.00	86.00	64.11		
AutoGen	Official	49.00	10.00	87.00	64.00	77.00	77.00	60.67	5.12	11.00
	Independent transfer	52.00	16.70	91.00	59.00	66.00	78.00	60.45		

Table 22: Illustrative process signals for the workflow wrappers in Main Experiment 1. Counts are from retained GAIA traces for the same wrappers under BenchAgent logging (Appendix Tables 13 and 14) and should be read as cross-experiment illustrative evidence, not matched broad-benchmark measurements or causal explanations.

Observed broad-result pattern	Illustrative wrapper-level signal	Comparable trace quant-ity	Bounded interpretation
EvoAgent has the only positive benchmark-balanced workflow lift, but its +1.44-point margin is within one-run uncertainty.	On retained GAIA runs, BenchAgent logs 19.13 Wiki/API requests and 0.12 tool errors per instance (Table 13).	The evolving wrapper varies workflow and prompt scaffolds inside a designer-specified search space.	The broad-suite gain is better read as task-specific scaffold recovery than a general multi-agent advantage.
EvoAgent’s largest gain is on BBH, while it trails on AIME, GSM8K, and HumanEval.	Retained GAIA logs record 0.39 code-agent steps per instance (Table 13).	Task-level traces separate where the workflow family helps from where it adds overhead.	Workflow search can locate a reasoning scaffold for some tasks but underperform when exact calculation dominates.
LLM-Debate leads on HumanEval and MATH but not on most other tasks.	BenchAgent logs 15.65 Wiki/API requests, 0.12 code-agent steps, and 0.02 code executions per instance (Table 13).	Independent proposal and aggregation stages are visible under the same evaluator.	Debate-style redundancy helps when alternatives can be checked locally, but is not a universal substitute for state preservation.
ChatEval is strongest on IFEval but has the highest token usage and trails the anchor overall.	Retained GAIA batches show 23.04 messages, 11.52 model calls, and 24.02 external-tool records per instance (Table 14).	Repeated discussion expands the message history under shared accounting.	Coordination can buy instruction-following robustness in one slice while erasing workflow lift overall.
Jarvis is faster and lighter but still loses accuracy.	Retained GAIA batches show 2.00 messages, 1.00 model call, and 0.13 direct/code markers per instance (Table 14).	Its predefined handoff exposes low coordination overhead.	Shallow coordination can be efficient but does not prevent missing task-specific reasoning modes.
Camel is faster and lighter but still loses accuracy.	BenchAgent logs 19.16 Wiki/API requests, 0.08 direct external-tool markers, and 0.06 code executions per instance (Table 13).	A low-overhead fixed-handoff pattern without the long discussion history of ChatEval.	Fixed handoff saves tokens but does not guarantee that the right evidence reaches the final decision stage.

Table 23: Evidence status for candidate mechanisms behind the CC-workflow GAIA result, with concrete ablation paths needed to resolve each claim.

Candidate mechanism	Observed trace evidence	Remaining confound	Ablation needed
Runtime delegation	Four structured dumps expose TeamCreate, Agent, and SendMessage events with 6.25 subagents on average.	No ablation disables delegation or fixes the agent count.	Compare no-subagent, fixed-agent-count, and adaptive-delegation variants under the same tools and answer contract.
Persistent artifacts	Retained traces include Read, Write, and evidence re-opening before finalization in some cases.	File tooling is bundled with the external runtime and task environment.	Disable writes or replace files with in-memory state while holding the solver and tool surface fixed.
Verifier-stage control	Two structured dumps show non-idle verifier emissions before final answer release.	The retained traces do not expose every recovery loop or failed verification path.	Ablate the verifier release gate or force final-answer emission before verification and measure error changes.
Context management	Some traces preserve task evidence across role boundaries and later checks.	Provider-side context compaction or bookkeeping is not fully observable.	Replay matched evidence packets with controlled context-window and compaction policies.

Table 24: Failure-step distribution from binary-search attribution on retained GAIA failed instances (GPT-4.1 backbone). Step index is the earliest conversation turn at which the agent output diverges from the ground truth. Jarvis (88 analyzed instances) commits errors overwhelmingly at step 1, consistent with its single-shot handoff. ChatEval (101 analyzed instances) also concentrates errors at step 1, showing that the debate loop does not reliably rescue initial-turn failures; a smaller fraction persists or is first introduced at later odd-numbered debate rounds.

Wrapper	Step 1	Step 3	Step 5	Step 7	Step 11	Step 13	Total analyzed
Jarvis	72 (81.8%)	—	—	—	—	—	88
ChatEval	58 (57.4%)	6 (5.9%)	2 (2.0%)	6 (5.9%)	1 (1.0%)	1 (1.0%)	101

Note: Jarvis step=0 (5 instances, 5.7%) and ChatEval unattributed instances are omitted. Step indices correspond to BenchAgent message-log positions.

Table 25: Structured-event statistics from retained Claude Code GAIA trajectory dumps with explicit team-creation and role-message structure.

Trace Dump	Subagents	Max Depth	Verifier Triggers	Explicit Repair Loops
3.json	5	1	0	0
6.json	4	4	3	0
claude_code_logs_gaia1.json	12	2	0	0
claude_code_logs_gaia3.json	4	3	2	0
Team-run average	6.25	2.50	1.25	0.00

Table 26: Team-initialization trace from 6.json item 17 (gaia-validation-l2p15). The team lead (CC-workflow orchestrator) emits one TeamCreate call followed by four Agent spawns in a single conversation turn, each with a distinct role prompt baked in at spawn time. Step numbers correspond to assistant message indices in the raw log.

Step	Actor	Tool / Event	Retained excerpt (parameters condensed)
1	Team Lead	TeamCreate	agent_type: planner; team_name: gaia-validation-l2p15; <i>description</i> : “Coordinated agent team for GAIA validation pipeline on level2_part15.jsonl. Each role operates with bash for file I/O; outputs only to outputs/ and logs/.”
	Runtime	spawn ack	agent_id: planner@gaia-validation-l2p15; team registered; agent awaiting mailbox.
2	Team Lead	Agent (planner)	model: sonnet; isolation: worktree; <i>prompt</i> : “Use bash tools ONLY (head, sed, tail) to read the first 3 lines from level2_part15.jsonl. For each, extract task_id and question, then pass them along with their line number (0-indexed) to solver, one at a time.”
3	Team Lead	Agent (solver)	model: sonnet; <i>prompt</i> : “You are the Solver. For each GAIA problem sent to you (task_id, question, line number), solve based strictly on the question and associated context. Prepare a concise answer in the required GAIA format. Send your answer, task_id, and reasoning to the Verifier.”
4	Team Lead	Agent (verifier)	model: sonnet; <i>prompt</i> : “For each problem: receive task_id, answer, and reasoning from Solver. Check GAIA answer format; assess answer validity. If correct and in format, forward to Writer with task_id, answer, reasoning. If not, return to Solver noting the issue.”
5	Team Lead	Agent (writer)	model: sonnet; <i>prompt</i> : “Upon receiving task_id, answer, and reasoning from the Verifier, check if outputs/<task_id>.txt exists. If it does, SKIP. If not, create outputs/<task_id>.txt and write ONLY the answer there. Always write the full reasoning log to logs/<task_id>.log.”
	Runtime	4× spawn ack	agent_id: solver@, verifier@, writer@gaia-validation-l2p15; all four roles live in shared mailbox.
6	Team Lead	Bash	head -n 3 split_validation/level2_part15.jsonl; first returned task_id: 48eb8242... (“How many non-indigenous crocodiles in Florida 2000–2020?”).

Table 27: Task-execution trace for CC-workflow instance 48eb8242 (GAIA Level 2, Florida crocodile count task, ground truth: 11). Steps E1–E11 span four role agents across items 15, 22, 24, and 30 of 6.json. Planner dispatches via a `SendMessage`; Solver resolves and forwards; Verifier checks format and routes; Writer enforces the skip-if-exists guard before committing. All role-level prompts were set at spawn time (Table 26).

Step	Role	Tool / Event	Retained excerpt
E1	Planner	Bash	<code>head -n 3 split_validation/level2_part15.jsonl</code> ; returns <code>task_id: 48eb8242...</code> , <i>Question</i> : “How many non-indigenous crocodiles were found in Florida from the year 2000 through 2020? You can get the data from the USGS Nonindigenous Aquatic Species database.”
E2	Planner	<code>SendMessage</code> (to solver)	Dispatches task to Solver’s mailbox: “Level2_part15.jsonl line 0: <code>task_id=48eb8242...</code> , <code>question=How many nonindigenous crocodiles...</code> ”; Solver receives as a teammate-message.
E3	Solver	[teammate-message from Planner]	Receives dispatch payload containing <code>task_id</code> and full question text; triggers Solver’s internal answer generation.
E4	Solver	[internal generation]	“Answer: 11. Reasoning: According to the USGS Nonindigenous Aquatic Species database, there were 11 confirmed occurrences of nonindigenous crocodiles in Florida between 2000 and 2020.”
E5	Solver	<code>SendMessage</code> (to Verifier)	<i>message</i> : “Answer: 11 / Task ID: 48eb8242... / Reasoning: USGS NAS database, 11 confirmed occurrences of nonindigenous <i>Crocodylus</i> in Florida 2000–2020.”; routing: sender=solver (green), target=@Verifier.
E6	Verifier	[teammate-message from Solver]	Receives <code>task_id</code> , answer, and reasoning; role instruction: “Check Gaia answer format; if correct and in format, forward to Writer; if not, return to Solver noting the issue.”
E7	Verifier	[format/validity check]	Answer “11” is a valid integer in GAIA format; no extra fields; reasoning is consistent. Proceeds to forward.
E8	Verifier	<code>SendMessage</code> (to writer)	<i>summary</i> : “Verified: answer and format for task 48eb8242...”; <i>content</i> : “Task ID: 48eb8242... / Answer: 11 / Reasoning: ...”; routing: sender=verifier (yellow), target=@writer (purple).
E9	Writer	[teammate-message from Verifier]	Receives: “Task ID: 48eb8242... / Answer: 11 / Reasoning: USGS NAS...”; role instruction: “check if <code>outputs/<task_id>.txt</code> exists; if so, SKIP; if not, write ONLY the answer.”
E10	Writer	<code>Read</code> (existence guard)	<code>file_path: outputs/48eb8242....txt</code> ; checks whether a prior write already committed this answer (skip-if-exists gate).
E11	Writer	<code>Write</code> (log commit)	<code>file_path: logs/48eb8242....log</code> ; writes full reasoning log; answer “11” written to <code>outputs/48eb8242....txt</code> . Task finalized.

Table 28: CC-workflow runtime spawn-time role prompts extracted verbatim from `6.json` item 17. Each prompt constitutes the full behavioral contract for the corresponding role; no further system-level instruction is issued after spawn. The prompts show role specialization (file-read restriction for Planner, format commitment for Solver, routing logic for Verifier, idempotent write guard for Writer) arising from a single live decision by the team lead, not from a pre-written template.

Role	Spawn-time prompt (verbatim)
Planner	Use bash tools ONLY (<code>head</code> , <code>sed</code> , <code>tail</code> , etc.) to read the first 3 lines from <code>split_validation/level2_part15.jsonl</code> . For each, extract the <code>task_id</code> and question, then pass them along with their line number (0-indexed) to solver, one at a time. Output nothing else. Do not use internal file reading functions.
Solver	You are the Solver. For each GAIA problem sent to you (<code>task_id</code> , question, line number), solve based strictly on the question and associated context. Prepare a concise answer string in the required GAIA answer format. Send your answer, <code>task_id</code> , and reasoning to the Verifier. Output nothing else.
Verifier	For each problem: receive <code>task_id</code> , answer, and reasoning from Solver. Check GAIA answer format (no extra data, correct fields, etc.); assess answer validity as best as possible. If correct and in format, forward to Writer with <code>task_id</code> , answer, reasoning. If not, return to Solver noting the issue. Output nothing else.
Writer	Upon receiving a <code>task_id</code> , answer, and reasoning from the verifier, check if <code>outputs/<task_id>.txt</code> exists. If it does, SKIP . If not, create <code>outputs/<task_id>.txt</code> and write ONLY the answer there, with no extra output. Always also write the full reasoning log to <code>logs/<task_id>.log</code> . Never overwrite existing files. Output nothing else.